

License

- Open Source: AGPLv3
- Commercial:

[LICENSE](#)

Go

```
go get github.com/mikhaildadaev/uast
```

uast

v1.26.11.

Run Test

```
go test ./...
go test -bench=. ./...
go test -cover ./...
go test -race ./...
```

Key Features

- —
- — AST MariaDB MsSQL MySQL PostgreSQL SQLite
- SQL
- — Constant / Literal / Value SQL
- — sync.Pool 216 ns/op
- 2 450 ns/op
- — Go
- — SQL
- — SetDialect()
- DDL — ALTER COMMENT CREATE DROP TRUNCATE
- DML — DELETE, INSERT, SELECT, UPDATE JOIN CTE
- UPSERT JSON

Limits

- MsSQLJSON CONTAINS / JSON TYPEMsSQL
- MySQLRETURNINGMySQL
- SQLiteRIGHT JOIN / RIGHT OUTER JOINSQLite

Supported Databases

Database	Version	Compatible
MariaDB	10.7.0+	DoltDB, SingleStore
MsSQL	16.0.0+	AmazonRDS, AzureSQL, Synapse
MySQL	8.0.31+	AuroraMySQL, AzureMySQL, GoogleMySQL, OceanBase, PlanetScale, TDSQL
PostgreSQL	9.5.0+	AlloyDB, ArenadataDB, AuroraPostgreSQL, AzurePostgreSQL, Citus, CockroachDB, GooglePostgreSQL, Greenplum, KingbaseES, Neon, OpenGauss, Supabase, TimescaleDB, YandexDB, YugabyteDB
SQLite	3.35.0+	CloudflareD1, LiteFS, Turso

SQL

API / Core /

NewAlterNewCommentNewCreateNewDelete

NewDropNewInsertNewSelectNewTruncateNewUpdate

Build()SQL

NewAlter

ALTERIndex/Schema/Table/View

AddColumnsAddConstraintsDropColumnsDropConstraintsRenameTo

RenameColumnRenameConstraintSetColumns

```

stmtAlterIndex := uast.NewAlter(uast.NewIndex("users_id", uast.NewTable("users'
    RenameTo("new_name")
stmtAlterSchema := uast.NewAlter(uast.NewTable("test")).
    RenameTo("new_name")
stmtAlterTableAddAndDrop := uast.NewAlter(uast.NewTable("users", "u")).
    AddColumns(
        Test.Table.Users.String,
        Test.Table.Users.Date,
    ).
    AddConstraints(
        Test.Check.OrdersNumber,
        Test.Primary.OrdersID,
        Test.Unique.OrdersName,
    ).
    DropColumns(
        Test.Table.Users.ID,
        Test.Table.Users.Name,
    ).
    DropConstraints(
        Test.Check.UsersNumber,
        Test.Foreign.UsersOrders,
        Test.Primary.UsersID,
        Test.Unique.UsersName,
    )
stmtAlterTableRenameColumn := uast.NewAlter(uast.NewTable("users", "u")).
    RenameColumn(uast.Field[int64]("u", "id"), "user_id")
stmtAlterTableRenameConstraint := uast.NewAlter(uast.NewTable("users", "u")).
    RenameConstraint(uast.NewCheck("ck_users_number", uast.Greater(uast.Field[
stmtAlterTableRenameTo := uast.NewAlter(uast.NewTable("users", "u")).
    RenameTo("new_users")
stmtAlterTableSet := uast.NewAlter(uast.NewTable("users", "u")).
    SetColumns(
        uast.Field[int64]("u", "id").SetType(uast.TypeUUID),
        uast.Field[string]("u", "string").SetDefault("_"),
        uast.Field[time.Time]("u", "date").SetNotNull(),
    )
stmtAlterView := uast.NewAlter(uast.NewView("users_general", "ug", uast.NewTab
    RenameTo("new_name")

```

Output MariaDB:

```

// Not supported
// Not supported
ALTER TABLE `users` ADD COLUMN `string` VARCHAR, ADD COLUMN `date` DATE, ADD C
ALTER TABLE `users` RENAME COLUMN `id` TO `user_id`
ALTER TABLE `users` RENAME CONSTRAINT `ck_users_number` TO `ck_user_number`
ALTER TABLE `users` RENAME TO `new_users`
ALTER TABLE `users` MODIFY COLUMN `id` UUID, ALTER COLUMN `string` SET DEFAULT
// Not supported

```

Output MsSQL:

```
// Not supported
// Not supported
ALTER TABLE [users] ADD COLUMN [string] NVARCHAR, ADD COLUMN [date] DATE, ADD C
// Not supported
// Not supported
// Not supported
ALTER TABLE [users] ALTER COLUMN [id] UNIQUEIDENTIFIER, ADD DEFAULT '_' FOR [s
// Not supported
```

Output MySQL:

```
// Not supported
// Not supported
ALTER TABLE `users` ADD COLUMN `string` VARCHAR, ADD COLUMN `date` DATE, ADD C
ALTER TABLE `users` RENAME COLUMN `id` TO `user_id`
ALTER TABLE `users` RENAME CONSTRAINT `ck_users_number` TO `ck_user_number`
ALTER TABLE `users` RENAME TO `new_users`
ALTER TABLE `users` MODIFY COLUMN `id` CHAR(36), ALTER COLUMN `string` SET DEF
// Not supported
```

Output PostgreSQL:

```
ALTER INDEX "users_id" RENAME TO "new_name"
ALTER SCHEMA "test" RENAME TO "new_name"
ALTER TABLE "users" ADD COLUMN "string" VARCHAR, ADD COLUMN "date" DATE, ADD C
ALTER TABLE "users" RENAME COLUMN "id" TO "user_id"
ALTER TABLE "users" RENAME CONSTRAINT "ck_users_number" TO "ck_user_number"
ALTER TABLE "users" RENAME TO "new_users"
ALTER TABLE "users" ALTER COLUMN "id" TYPE UUID, ALTER COLUMN "string" SET DEF
ALTER VIEW "users_general" RENAME TO "new_name"
```

Output SQLite:

```
// Not supported
// Not supported
ALTER TABLE "users" ADD COLUMN "string" TEXT, ADD COLUMN "date" TEXT
// Not supported
// Not supported
ALTER TABLE "users" RENAME TO "new_users"
ALTER TABLE "users" ALTER COLUMN "date" NOT NULL
// Not supported
```

NewComment

COMMENT

OnColumn

Is

```
stmtCommentColumn := uast.NewComment(uast.NewTable("users", "u")).OnColumn(uast.NewTable("users", "u")).Is("text")
stmtCommentTable := uast.NewComment(uast.NewTable("users", "u")).Is("text")
```

Output MariaDB:

```
COMMENT ON COLUMN `users`.`id` IS 'text'
COMMENT ON TABLE `users` AS `u` IS 'text'
```

Output MsSQL:

```
// Not supported
// Not supported
```

Output MySQL:

```
COMMENT ON COLUMN `users`.`id` IS 'text'
COMMENT ON TABLE `users` AS `u` IS 'text'
```

Output PostgreSQL:

```
COMMENT ON COLUMN "users"."id" IS 'text'
COMMENT ON TABLE "users" AS "u" IS 'text'
```

Output SQLite:

```
COMMENT ON COLUMN "users"."id" IS 'text'
COMMENT ON TABLE "users" AS "u" IS 'text'
```

NewCreate

CREATE

Index/Schema/Table/View

Columns

Constraints

IfNotExists

IsReplace

IsUnique

On

Source

```

stmtCreateIndex := uast.NewCreate(uast.NewIndex("users_id", uast.NewTable("use
    IfNotExists().
    IsUnique().
    On(uast.NewTable("users", "u")).
    Columns(
        uast.Field[string]("u", "string"),
        uast.Field[int]("u", "number"),
    )
stmtCreateScheme := uast.NewCreate(uast.NewSchema("test")).
    IfNotExists()
stmtCreateTable := uast.NewCreate(uast.NewTable("users", "u")).
    Constraints(
        uast.NewCheck("ck_users_number", uast.Greater(uast.Field[int]("u", "nu
        uast.NewForeignKey("fk_users_orders", uast.NewTable("orders", "o"), ua
            uast.Relation(uast.Field[int64]("u", "order_id"), uast.Field[int64
            uast.Relation(uast.Field[string]("u", "name"), uast.Field[string](
        ),
        uast.NewPrimaryKey("pk_users_id", uast.Field[int64]("u", "id")),
        uast.NewUnique("un_users_name", uast.Field[string]("u", "name")),
    ).
    IfNotExists().
    Columns(
        uast.Field[int64]("u", "id").AutoIncrement(),
        uast.Field[string]("u", "name").NotNull(),
    )
stmtCreateView := uast.NewCreate(uast.NewView("users_general", "ug", uast.NewT
    IsReplace().
    Source(uast.NewSelect(uast.NewTable("users", "u")).
        Fields(
            uast.Field[int64]("u", "id"),
            uast.Field[string]("u", "string"),
        ),
    )

```

Output MariaDB:

```

CREATE UNIQUE INDEX IF NOT EXISTS `users_id` ON `users` (`string`, `number`)
CREATE SCHEMA IF NOT EXISTS `test`
CREATE TABLE IF NOT EXISTS `users` (`id` SIGNED AUTO_INCREMENT, `name` VARCHAR
CREATE OR REPLACE VIEW `users_general` AS SELECT `u`.`id`, `u`.`string` FROM `

```

Output MsSQL:

```
CREATE UNIQUE INDEX [users_id] ON [users] ([string], [number])
CREATE SCHEMA IF NOT EXISTS [test]
CREATE TABLE [users] ([id] BIGINT IDENTITY(1,1), [name] NVARCHAR NOT NULL, CON
CREATE OR REPLACE VIEW [users_general] AS SELECT [u].[id], [u].[string] FROM [u]
```

Output MySQL:

```
CREATE UNIQUE INDEX `users_id` ON `users` (`string`, `number`)
CREATE SCHEMA `test`
CREATE TABLE IF NOT EXISTS `users` (`id` SIGNED AUTO_INCREMENT, `name` VARCHAR
CREATE OR REPLACE VIEW `users_general` AS SELECT `u`.`id`, `u`.`string` FROM `u`
```

Output PostgreSQL:

```
CREATE UNIQUE INDEX IF NOT EXISTS "users_id" ON "users" ("string", "number")
CREATE SCHEMA IF NOT EXISTS "test"
CREATE TABLE IF NOT EXISTS "users" ("id" BIGINT GENERATED BY DEFAULT AS IDENTI
CREATE OR REPLACE VIEW "users_general" AS SELECT "u"."id", "u"."string" FROM "u"
```

Output SQLite:

```
CREATE UNIQUE INDEX IF NOT EXISTS "users_id" ON "users" ("string", "number")
// Not supported
CREATE TABLE IF NOT EXISTS "users" ("id" INTEGER AUTOINCREMENT, "name" TEXT NO
CREATE OR REPLACE VIEW "users_general" AS SELECT "u"."id", "u"."string" FROM "u"
```

NewDelete

DELETE

Join

Returning

Where

With

```

stmtDeleteJoin := uast.NewDelete(uast.NewTable("users", "u")).
    Join(
        uast.Inner(uast.NewTable("orders", "o"), uast.Equal(uast.Field[int64]("o", "id"), uast.Field[int64]("u", "id"))),
    ).
    Where(
        uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),
    )
stmtDeleteReturning := uast.NewDelete(uast.NewTable("users", "u")).
    Where(
        uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    Returning(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string"),
    )
stmtDeleteWhere := uast.NewDelete(uast.NewTable("users", "u")).
    Where(
        uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),
    )
stmtDeleteWith := NewDelete(uast.NewTable("users", "u")).
    With(
        uast.WithN("old_users", uast.NewSelect(uast.NewTable("users", "u")).
            Field(
                uast.Field[int64]("u", "id"),
            ).
            Where(
                Less(uast.Field[int]("u", "number"), Value(2)),
            ),
        ),
    )

```

Output MariaDB:

```

DELETE `u` FROM `users` AS `u` INNER JOIN `orders` AS `o` ON `u`.`id` = `o`.`id`
DELETE `u` FROM `users` AS `u` WHERE `u`.`string` = ? RETURNING `u`.`id`, `u`.`string`
DELETE `u` FROM `users` AS `u` WHERE `u`.`string` = ?
WITH `old_users` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` < 2)

```

Output MsSQL:

```

DELETE [u] FROM [users] AS [u] INNER JOIN [orders] AS [o] ON [u].[id] = [o].[id]
DELETE [u] FROM [users] AS [u] **Output [u].[id], [u].[string] WHERE [u].[string] = @p1
DELETE [u] FROM [users] AS [u] WHERE [u].[string] = @p1
WITH [old_users] AS (SELECT [u].[id] FROM [users] AS [u] WHERE [u].[number] < 2)

```

Output MySQL:

```
DELETE `u` FROM `users` AS `u` INNER JOIN `orders` AS `o` ON `u`.`id` = `o`.`id`  
DELETE `u` FROM `users` AS `u` WHERE `u`.`string` = ?  
DELETE `u` FROM `users` AS `u` WHERE `u`.`string` = ?  
WITH `old_users` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` <
```

Output PostgreSQL:

```
DELETE FROM "users" AS "u" USING "orders" AS "o" WHERE ("u"."id" = "o"."id" AND  
DELETE FROM "users" AS "u" WHERE "u"."string" = $1 RETURNING "u"."id", "u"."st  
DELETE FROM "users" AS "u" WHERE "u"."string" = $1  
WITH "old_users" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" <
```

Output SQLite:

```
DELETE FROM "users" AS "u" INNER JOIN "orders" AS "o" ON "u"."id" = "o"."id" W  
DELETE FROM "users" AS "u" WHERE "u"."string" = ? RETURNING "u"."id", "u"."str  
DELETE FROM "users" AS "u" WHERE "u"."string" = ?  
WITH "old_users" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" <
```

NewDrop

DROP

Index/Schema/Table/View

Cascade()

IfExists()

```

stmtDropCascadeIndex := uast.NewDrop(uast.NewIndex("users_id", uast.NewTable("
    Cascade()
stmtDropCascadeSchema := uast.NewDrop(uast.NewSchema("test")).
    Cascade()
stmtDropCascadeTable := uast.NewDrop(uast.NewTable("users", "u")).
    Cascade()
stmtDropCascadeView := uast.NewDrop(uast.NewView("users_general", "ug", uast.N
    Cascade()
stmtDropIfExistsIndex := uast.NewDrop(uast.NewIndex("users_id", uast.NewTable('
    IfExists()
stmtDropIfExistsSchema := uast.NewDrop(uast.NewSchema("test")).
    IfExists()
stmtDropIfExistsTable := uast.NewDrop(uast.NewTable("users", "u")).
    IfExists()
stmtDropIfExistsView := uast.NewDrop(uast.NewView("users_general", "ug", uast.I
    IfExists()

```

Output MariaDB:

```

DROP INDEX `users_id` CASCADE
DROP SCHEMA `test` CASCADE
DROP TABLE `users`
DROP VIEW `users_general` CASCADE
DROP INDEX IF EXISTS `users_id`
DROP SCHEMA IF EXISTS `test`
DROP TABLE IF EXISTS `users`
DROP VIEW IF EXISTS `users_general`

```

Output MsSQL:

```

DROP INDEX [users_id]
DROP SCHEMA [test]
DROP TABLE [users]
DROP VIEW [users_general]
DROP INDEX [users_id]
DROP SCHEMA IF EXISTS [test]
DROP TABLE IF EXISTS [users]
DROP VIEW IF EXISTS [users_general]

```

Output MySQL:

```
DROP INDEX `users_id`  
DROP SCHEMA `test`  
DROP TABLE `users`  
DROP VIEW `users_general`  
DROP INDEX IF EXISTS `users_id`  
DROP SCHEMA `test`  
DROP TABLE IF EXISTS `users`  
DROP VIEW IF EXISTS `users_general`
```

Output PostgreSQL:

```
DROP INDEX "users_id" CASCADE  
DROP SCHEMA "test" CASCADE  
DROP TABLE "users" CASCADE  
DROP VIEW "users_general" CASCADE  
DROP INDEX IF EXISTS "users_id"  
DROP SCHEMA IF EXISTS "test"  
DROP TABLE IF EXISTS "users"  
DROP VIEW IF EXISTS "users_general"
```

Output SQLite:

```
DROP INDEX "users_id"  
DROP SCHEMA "test"  
DROP TABLE "users"  
DROP VIEW "users_general"  
DROP INDEX IF EXISTS "users_id"  
DROP SCHEMA "test"  
DROP TABLE IF EXISTS "users"  
DROP VIEW IF EXISTS "users_general"
```

NewInsert

INSERT

Returning

Source/Values

With

```

stmtInsertReturning := uast.NewInsert(uast.NewTable("users", "u")).
    Values(
        uast.Pair(uast.Field[string]("u", "string"), uast.Value("ivan")),
        uast.Pair(uast.Field[int]("u", "number"), uast.Value(2)),
    ).
    Returning(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string"),
    )
stmtInsertSource := uast.NewInsert(uast.NewTable("users", "u")).
    Source(NewSelect(uast.NewTable("users", "u")).
        Field(
            uast.Field[string]("u", "string"),
            uast.Field[int]("u", "number"),
        ).
        Where(
            uast.Equal(uast.Field[string]("u", "string"), uast.Value("active"))
        ),
    )
stmtInsertValues := uast.NewInsert(uast.NewTable("users", "u")).
    Values(
        uast.Pair(uast.Field[string]("u", "string"), uast.Value("ivan")),
        uast.Pair(uast.Field[int]("u", "number"), uast.Value(2)),
    ).
    Upsert(
        uast.Pair(uast.Field[string]("u", "string"), uast.Value("updated")),
    )
stmtInsertWith := NewInsert(uast.NewTable("users", "u")).
    Values(
        uast.Pair(uast.Field[string]("u", "string"), uast.Value("ivan")),
        uast.Pair(uast.Field[int]("u", "number"), uast.Value(2)),
    ).
    With(
        uast.WithN("old_users", uast.NewSelect(uast.NewTable("users", "u")).
            Field(
                uast.Field[int64]("u", "id"),
            ).
            Where(
                uast.Less(uast.Field[int]("u", "number"), uast.Value(2)),
            ),
        ),
    )

```

Output MariaDB:

```

INSERT INTO `users` AS `u` (`string`, `number`) VALUES (?, ?) RETURNING `u`.`id`
INSERT INTO `users` AS `u` (`string`, `number`) SELECT `u`.`string`, `u`.`number`
INSERT INTO `users` AS `u` (`string`, `number`) VALUES (?, ?) ON DUPLICATE KEY
WITH `old_users` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` <

```

Output MsSQL:

```
INSERT INTO [users] AS [u] ([string], [number]) **Output [u].[id], [u].[string]
INSERT INTO [users] AS [u] ([string], [number]) SELECT [u].[string], [u].[number]
INSERT INTO [users] AS [u] ([string], [number]) VALUES (@p1, @p2)
WITH [old_users] AS (SELECT [u].[id] FROM [users] AS [u] WHERE [u].[number] < 100)
```

Output MySQL:

```
INSERT INTO `users` AS `u` (`string`, `number`) VALUES (?, ?)
INSERT INTO `users` AS `u` (`string`, `number`) SELECT `u`.`string`, `u`.`number`
INSERT INTO `users` AS `u` (`string`, `number`) VALUES (?, ?) ON DUPLICATE KEY UPDATE
WITH `old_users` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` < 100)
```

Output PostgreSQL:

```
INSERT INTO "users" AS "u" ("string", "number") VALUES ($1, $2) RETURNING "u"."id"
INSERT INTO "users" AS "u" ("string", "number") SELECT "u"."string", "u"."number"
INSERT INTO "users" AS "u" ("string", "number") VALUES ($1, $2) ON CONFLICT DO UPDATE
WITH "old_users" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" < 100)
```

Output SQLite:

```
INSERT INTO "users" AS "u" ("string", "number") VALUES (?, ?) RETURNING "u"."id"
INSERT INTO "users" AS "u" ("string", "number") SELECT "u"."string", "u"."number"
INSERT INTO "users" AS "u" ("string", "number") VALUES (?, ?) ON CONFLICT DO UPDATE
WITH "old_users" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" < 100)
```

NewSelect

SELECT Distinct Field GroupBy Having Join OrderBy Pagination Unions Where With

```

stmtSelectDistinct := uast.NewSelect(uast.NewTable("users", "u")).
    Distinct().
    Field(
        uast.Field[int64]("u", "id"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    )
stmtSelectField := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[int64]("u", "id"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    )
stmtSelectGroupBy := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[string]("u", "string"),
        uast.Count(uast.Field[int64]("u", "id"), false).As("cnt"),
    ).
    GroupBy(
        uast.Field[string]("u", "string"),
    )
stmtSelectHaving := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[string]("u", "string"),
        uast.Count(uast.Field[int64]("u", "id"), false).As("cnt"),
    ).
    GroupBy(
        uast.Field[string]("u", "string"),
    ).
    Having(
        uast.Greater(uast.Count(uast.Field[int64]("u", "id"), false), uast.Val
    )
stmtSelectJoin := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("o", "string"),
    ).
    Join(
        uast.Inner(uast.NewTable("orders", "o"), Equal(uast.Field[int64]("u",
    )
stmtSelectOrderBy := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[int64]("u", "id"),
    ).
    OrderBy(
        uast.Desc(uast.Field[int]("u", "number")),
        uast.Asc(uast.Field[string]("u", "string")),
    )
stmtSelectPagination := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[int64]("u", "id"),
    ).

```

```

    Pagination(10, 20)
stmtSelectUnions := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[string]("u", "string"),
    ).
    Unions(
        uast.UnionAll(uast.NewSelect(uast.NewTable("orders", "o")).
            Field(
                uast.Field[string]("o", "string"),
            ),
        ),
    )
stmtSelectWhere := uast.NewSelect(uast.NewTable("users", "u")).
    Field(
        uast.Field[int64]("u", "id"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    )
stmtSelectWith := uast.NewSelect(uast.NewCTE("cte_test", "ct")).
    Field(
        uast.Field[int64]("ct", "id"),
    ).
    With(
        uast.WithN("cte_test", uast.NewSelect(uast.NewTable("users", "u")).
            Field(
                uast.Field[int64]("u", "id"),
            ).
            Where(
                uast.Greater(uast.Field[int]("u", "number"), uast.Value(2)),
            ),
        ),
    )

```

Output MariaDB:

```

SELECT DISTINCT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` = ?
SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` = ?
SELECT `u`.`string`, COUNT(`u`.`id`) AS `cnt` FROM `users` AS `u` GROUP BY `u`.`string`
SELECT `u`.`string`, COUNT(`u`.`id`) AS `cnt` FROM `users` AS `u` GROUP BY `u`.`string`
SELECT `u`.`id`, `o`.`string` FROM `users` AS `u` INNER JOIN `orders` AS `o` ON `u`.`id` = `o`.`id`
SELECT `u`.`id` FROM `users` AS `u` ORDER BY `u`.`number` DESC, `u`.`string` ASC
SELECT `u`.`id` FROM `users` AS `u` LIMIT ? OFFSET ?
SELECT `u`.`string` FROM `users` AS `u` UNION ALL SELECT `o`.`string` FROM `orders` AS `o`
SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` = ?
WITH `cte_test` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` > ?)

```

Output MsSQL:

```

SELECT DISTINCT [u].[id] FROM [users] AS [u] WHERE [u].[number] = @p1
SELECT [u].[id] FROM [users] AS [u] WHERE [u].[number] = @p1
SELECT [u].[string], COUNT([u].[id]) AS [cnt] FROM [users] AS [u] GROUP BY [u].[string]
SELECT [u].[string], COUNT([u].[id]) AS [cnt] FROM [users] AS [u] GROUP BY [u].[string]
SELECT [u].[id], [o].[string] FROM [users] AS [u] INNER JOIN [orders] AS [o] ON [u].[id] = [o].[id]
SELECT [u].[id] FROM [users] AS [u] ORDER BY [u].[number] DESC, [u].[string] ASC
SELECT [u].[id] FROM [users] AS [u] ORDER BY 1 ASC OFFSET @p1 ROWS FETCH NEXT @p2 ROWS ONLY
SELECT [u].[string] FROM [users] AS [u] UNION ALL SELECT [o].[string] FROM [orders] AS [o]
SELECT [u].[id] FROM [users] AS [u] WHERE [u].[number] = @p1
WITH [cte_test] AS (SELECT [u].[id] FROM [users] AS [u] WHERE [u].[number] > @p1)

```

Output MySQL:

```

SELECT DISTINCT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` = ?
SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` = ?
SELECT `u`.`string`, COUNT(`u`.`id`) AS `cnt` FROM `users` AS `u` GROUP BY `u`.`string`
SELECT `u`.`string`, COUNT(`u`.`id`) AS `cnt` FROM `users` AS `u` GROUP BY `u`.`string`
SELECT `u`.`id`, `o`.`string` FROM `users` AS `u` INNER JOIN `orders` AS `o` ON `u`.`id` = `o`.`id`
SELECT `u`.`id` FROM `users` AS `u` ORDER BY `u`.`number` DESC, `u`.`string` ASC
SELECT `u`.`id` FROM `users` AS `u` LIMIT ? OFFSET ?
SELECT `u`.`string` FROM `users` AS `u` UNION ALL SELECT `o`.`string` FROM `orders` AS `o`
SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` = ?
WITH `cte_test` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` > ?)

```

Output PostgreSQL:

```

SELECT DISTINCT "u"."id" FROM "users" AS "u" WHERE "u"."number" = $1
SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" = $1
SELECT "u"."string", COUNT("u"."id") AS "cnt" FROM "users" AS "u" GROUP BY "u"."string"
SELECT "u"."string", COUNT("u"."id") AS "cnt" FROM "users" AS "u" GROUP BY "u"."string"
SELECT "u"."id", "o"."string" FROM "users" AS "u" INNER JOIN "orders" AS "o" ON "u"."id" = "o"."id"
SELECT "u"."id" FROM "users" AS "u" ORDER BY "u"."number" DESC, "u"."string" ASC
SELECT "u"."id" FROM "users" AS "u" LIMIT $1 OFFSET $2
SELECT "u"."string" FROM "users" AS "u" UNION ALL SELECT "o"."string" FROM "orders" AS "o"
SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" = $1
WITH "cte_test" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" > $1)

```

Output SQLite:

```

SELECT DISTINCT "u"."id" FROM "users" AS "u" WHERE "u"."number" = ?
SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" = ?
SELECT "u"."string", COUNT("u"."id") AS "cnt" FROM "users" AS "u" GROUP BY "u"
SELECT "u"."string", COUNT("u"."id") AS "cnt" FROM "users" AS "u" GROUP BY "u"
SELECT "u"."id", "o"."string" FROM "users" AS "u" INNER JOIN "orders" AS "o" ON
SELECT "u"."id" FROM "users" AS "u" ORDER BY "u"."number" DESC, "u"."string" ASC
SELECT "u"."id" FROM "users" AS "u" LIMIT ? OFFSET ?
SELECT "u"."string" FROM "users" AS "u" UNION ALL SELECT "o"."string" FROM "orders" AS "o"
SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" = ?
WITH "cte_test" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" > ?)

```

NewTruncate

TRUNCATE

Cascade()

RestartIdentity()

```

stmtTruncateDefault := uast.NewTruncate(uast.NewTable("users", "u"))
stmtTruncateCascade := uast.NewTruncate(uast.NewTable("users", "u")).
    Cascade()
stmtTruncateRestartIdentity := uast.NewTruncate(uast.NewTable("users", "u")).
    RestartIdentity()

```

Output MariaDB:

```

TRUNCATE TABLE `users`
TRUNCATE TABLE `users` CASCADE
TRUNCATE TABLE `users` RESTART IDENTITY

```

Output MsSQL:

```

TRUNCATE TABLE [users]
// Not supported
// Not supported

```

Output MySQL:

```
TRUNCATE TABLE `users`  
// Not supported  
// Not supported
```

Output PostgreSQL:

```
TRUNCATE TABLE "users"  
TRUNCATE TABLE "users" CASCADE  
TRUNCATE TABLE "users" RESTART IDENTITY
```

Output SQLite:

```
TRUNCATE TABLE "users"  
// Not supported  
// Not supported
```

NewUpdate

UPDATE

Where

With

Join

Returning

Set

```

stmtUpdateJoin := uast.NewUpdate(uast.NewTable("users", "u")).
    Join(
        uast.Inner(uast.NewTable("orders", "o"), Equal(uast.Field[int64]("u", '
    ).
    Set(
        uast.Assign(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    ).
stmtUpdateReturning := uast.NewUpdate(uast.NewTable("users", "u")).
    Set(
        uast.Assign(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    ).
    Returning(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string")
    )
stmtUpdateSet := uast.NewUpdate(uast.NewTable("users", "u")).
    Set(
        uast.Assign(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    )
stmtUpdateWhere := uast.NewUpdate(uast.NewTable("users", "u")).
    Set(
        uast.Assign(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "number"), uast.Value(2)),
    )
stmtUpdateWith := NewUpdate(uast.NewTable("users", "u")).
    Set(
        uast.Assign(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    With(
        uast.WithN("old_users", uast.NewSelect(uast.NewTable("users", "u")).
            Field(
                uast.Field[int64]("u", "id"),
            ).
            Where(
                uast.Less(uast.Field[int]("u", "number"), uast.Value(2)),
            ),
        ),
    )

```

Output MariaDB:

```

UPDATE `users` AS `u` INNER JOIN `orders` AS `o` ON `u`.`id` = `o`.`id` SET `u`
UPDATE `users` AS `u` SET `u`.`string` = ? WHERE `u`.`number` = ? RETURNING `u`
UPDATE `users` AS `u` SET `u`.`string` = ? WHERE `u`.`number` = ?
UPDATE `users` AS `u` SET `u`.`string` = ? WHERE `u`.`number` = ?
WITH `old_users` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` <

```

Output MsSQL:

```

UPDATE [users] AS [u] INNER JOIN [orders] AS [o] ON [u].[id] = [o].[id] SET [u]
UPDATE [users] AS [u] **Output [u].[id], [u].[string] SET [u].[string] = @p1 W
UPDATE [users] AS [u] SET [u].[string] = @p1 WHERE [u].[number] = @p2
UPDATE [users] AS [u] SET [u].[string] = @p1 WHERE [u].[number] = @p2
WITH [old_users] AS (SELECT [u].[id] FROM [users] AS [u] WHERE [u].[number] < (

```

Output MySQL:

```

UPDATE `users` AS `u` INNER JOIN `orders` AS `o` ON `u`.`id` = `o`.`id` SET `u`
UPDATE `users` AS `u` SET `u`.`string` = ? WHERE `u`.`number` = ?
UPDATE `users` AS `u` SET `u`.`string` = ? WHERE `u`.`number` = ?
UPDATE `users` AS `u` SET `u`.`string` = ? WHERE `u`.`number` = ?
WITH `old_users` AS (SELECT `u`.`id` FROM `users` AS `u` WHERE `u`.`number` <

```

Output PostgreSQL:

```

UPDATE "users" AS "u" INNER JOIN "orders" AS "o" ON "u"."id" = "o"."id" SET "u"
UPDATE "users" AS "u" SET "u"."string" = $1 WHERE "u"."number" = $2 RETURNING "u"
UPDATE "users" AS "u" SET "u"."string" = $1 WHERE "u"."number" = $2
UPDATE "users" AS "u" SET "u"."string" = $1 WHERE "u"."number" = $2
WITH "old_users" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" <

```

Output SQLite:

```

UPDATE "users" AS "u" INNER JOIN "orders" AS "o" ON "u"."id" = "o"."id" SET "u"
UPDATE "users" AS "u" SET "u"."string" = ? WHERE "u"."number" = ? RETURNING "u"
UPDATE "users" AS "u" SET "u"."string" = ? WHERE "u"."number" = ?
UPDATE "users" AS "u" SET "u"."string" = ? WHERE "u"."number" = ?
WITH "old_users" AS (SELECT "u"."id" FROM "users" AS "u" WHERE "u"."number" <

```

API / Core /

SQL

As

Over

exprColumn

As

```
column := uast.Field[string]("u", "string").As("alias")
```

Output MariaDB:

```
`u`.`string` AS `alias`
```

Output MsSQL:

```
[u].[string] AS [alias]
```

Output MySQL:

```
`u`.`string` AS `alias`
```

Output PostgreSQL:

```
"u"."string" AS "alias"
```

Output SQLite:

```
"u"."string" AS "alias"
```

exprFunction

As

```
function := uast.Avg(uast.Field[int]("u", "number"), false).As("alias")
```

Output MariaDB:

```
AVG(`u`.`number`) AS `alias`
```

Output MsSQL:

```
AVG([u].[number]) AS [alias]
```

Output MySQL:

```
AVG(`u`.`number`) AS `alias`
```

Output PostgreSQL:

```
AVG("u"."number") AS "alias"
```

Output SQLite:

```
AVG("u"."number") AS "alias"
```

Over

```
function := uast.Avg(uast.Field[int]("u", "number"), false).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
AVG(`u`.`number`) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
AVG([u].[number]) OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
AVG(`u`.`number`) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
AVG("u"."number") OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
AVG("u"."number") OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

exprSubquery

As

```
subquery := uast.Subquery[int64](uast.NewSelect(uast.Field[int64]("u", "id")).
```

Output MariaDB:

```
(SELECT `u`.`id` FROM `users` AS `u`) AS `alias`
```

Output MsSQL:

```
(SELECT [u].[id] FROM [users] AS [u]) AS [alias]
```

Output MySQL:

```
(SELECT `u`.`id` FROM `users` AS `u`) AS `alias`
```

Output PostgreSQL:

```
(SELECT "u"."id" FROM "users" AS "u") AS "alias"
```

Output SQLite:

```
(SELECT "u"."id" FROM "users" AS "u") AS "alias"
```

API / Core /

`clauseGroupBy` `clauseHaving` `clauseJoin`
`clauseOrderBy` `clausePagination` `clauseReturning` `clauseSet` `clauseUnions`
`clauseValues` `clauseWhere` `clauseWith` `exprArray` `exprBinary`
`exprComparison` `exprConstant` `exprField` `exprFunction` `exprLiteral`
`exprLogical` `exprSubquery` `exprValue`

clauseGroupBy

GROUP BY

```
groupBy := GroupBy(  
    uast.Field[string]("u", "string"),  
)
```

Output MariaDB:

```
GROUP BY `u`.`string`
```

Output MsSQL:

```
GROUP BY [u].[string]
```

Output MySQL:

```
GROUP BY `u`.`string`
```

Output PostgreSQL:

```
GROUP BY "u"."string"
```

Output SQLite:

```
GROUP BY "u"."string"
```

clauseHaving

HAVING

GROUP BY

```
having := Having(  
    uast.Greater(uast.Count(uast.Field[int64]("u", "id"), false), uast.Value[i  
)
```

Output MariaDB:

```
HAVING COUNT(`u`.`id`) > ?
```

Output MsSQL:

```
HAVING COUNT([u].[id]) > @p1
```

Output MySQL:

```
HAVING COUNT(`u`.`id`) > ?
```

Output PostgreSQL:

```
HAVING COUNT("u"."id") > $1
```

Output SQLite:

```
HAVING COUNT("u"."id") > ?
```

clauseJoin

Cross

CROSS JOIN

```
join := uast.Cross(uast.NewTable("users").As("u"))
```

Output MariaDB:

```
CROSS JOIN `users` AS `u`
```

Output MsSQL:

```
CROSS JOIN [users] AS [u]
```

Output MySQL:

```
CROSS JOIN `users` AS `u`
```

Output PostgreSQL:

```
CROSS JOIN "users" AS "u"
```

Output SQLite:

```
CROSS JOIN "users" AS "u"
```

Full

FULL JOIN

NULL

```
join := uast.Full(uast.NewTable("users").As("u"), uast.Equal(uast.Field[int64]
```

Output MariaDB:

```
FULL JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
FULL JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
FULL JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
FULL JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
FULL JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

FullOuter

FULL OUTER JOIN

NULL

```
join := uast.FullOuter(uast.NewTable("users").As("u"), uast.Equal(uast.Field[i
```

Output MariaDB:

```
FULL OUTER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
FULL OUTER JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
FULL OUTER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
FULL OUTER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
FULL OUTER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Inner

INNER JOIN

```
join := uast.Inner(uast.NewTable("users").As("u"), uast.Equal(uast.Field[int64
```

Output MariaDB:

```
INNER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
INNER JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
INNER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
INNER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
INNER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Left

LEFT JOIN

```
join := uast.Left(uast.NewTable("users").As("u"), uast.Equal(uast.Field[int64]
```

Output MariaDB:

```
LEFT JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
LEFT JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
LEFT JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
LEFT JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
LEFT JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

LeftOuter

LEFT OUTER JOIN

```
join := uast.LeftOuter(uast.NewTable("users").As("u"), uast.Equal(uast.Field[i
```

Output MariaDB:

```
LEFT OUTER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
LEFT OUTER JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
LEFT OUTER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
LEFT OUTER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
LEFT OUTER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Right

RIGHT JOIN

SQLite

```
join := uast.Right(uast.NewTable("users").As("u"), uast.Equal(uast.Field[int64
```

Output MariaDB:

```
RIGHT JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
RIGHT JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
RIGHT JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
RIGHT JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
// Not supported
```

RightOuter

RIGHT OUTER JOIN

SQLite

```
join := uast.RightOuter(uast.NewTable("users").As("u"), uast.Equal(uast.Field[
```

Output MariaDB:

```
RIGHT OUTER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output MsSQL:

```
RIGHT OUTER JOIN [users] AS [u] ON [u].[id] = [t1].[id]
```

Output MySQL:

```
RIGHT OUTER JOIN `users` AS `u` ON `u`.`id` = `t1`.`id`
```

Output PostgreSQL:

```
RIGHT OUTER JOIN "users" AS "u" ON "u"."id" = "t1"."id"
```

Output SQLite:

```
// Not supported
```

clauseOrderBy

Asc

A Z

```
orderBy := uast.Asc(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
`u`.`string` ASC
```

Output MsSQL:

```
[u].[string] ASC
```

Output MySQL:

```
`u`.`string` ASC
```

Output PostgreSQL:

```
"u"."string" ASC
```

Output SQLite:

```
"u"."string" ASC
```

Desc

Z A

```
orderBy := uast.Desc(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
`u`.`string` DESC
```

Output MsSQL:

```
[u].[string] DESC
```

Output MySQL:

```
`u`.`string` DESC
```

Output PostgreSQL:

```
"u"."string" DESC
```

Output SQLite:

```
"u"."string" DESC
```

clausePagination

```
Pagination(limit, offset)    SELECT    limit    offset
```

```
pagination := Pagination(10,0)
```

Output MariaDB:

```
LIMIT ? OFFSET ?
```

Output MsSQL:

```
OFFSET @p1 ROWS FETCH NEXT @p2 ROWS ONLY
```

Output MySQL:

```
LIMIT ? OFFSET ?
```

Output PostgreSQL:

```
LIMIT $1 OFFSET $2
```

Output SQLite:

```
LIMIT ? OFFSET ?
```

clauseReturning

RETURNING

MariaDB

PostgreSQL

SQLite

MySQL

```
returning = Returning(  
    uast.Field[int64]("u", "id"),  
    uast.Field[string]("u", "string"),  
)
```

Output MariaDB:

```
RETURNING `u`.`id`, `u`.`string`
```

Output MsSQL:

```
**Output [u].[id], [u].[string]
```

Output MySQL:

```
// Not support
```

Output PostgreSQL:

```
RETURNING "u"."id", "u"."string"
```

Output SQLite:

```
RETURNING "u"."id", "u"."string"
```

clauseSet

Assign

Assign

```
set := Set(  
    uast.Assign(uast.Field[string]("u", "string"), uast.Value("active")),  
)
```

Output MariaDB:

```
UPDATE `users` AS `u` SET `u`.`string` = ?
```

Output MsSQL:

```
UPDATE [users] AS [u] SET [u].[string] = @p1
```

Output MySQL:

```
UPDATE `users` AS `u` SET `u`.`string` = ?
```

Output PostgreSQL:

```
UPDATE "users" AS "u" SET "u"."string" = $1
```

Output SQLite:

```
UPDATE "users" AS "u" SET "u"."string" = ?
```

clauseUnions

Union

SELECT UNION

```
unions := uast.Union(uast.NewSelect(uast.NewTable("users").As("u")).  
    Field(  
        uast.Field[string]("u", "string"),  
    ),  
)
```

Output MariaDB:

```
UNION SELECT `u`.`string` FROM `users` AS `u`
```

Output MsSQL:

```
UNION SELECT [u].[string] FROM [users] AS [u]
```

Output MySQL:

```
UNION SELECT `u`.`string` FROM `users` AS `u`
```

Output PostgreSQL:

```
UNION SELECT "u"."string" FROM "users" AS "u"
```

Output SQLite:

```
UNION SELECT "u"."string" FROM "users" AS "u"
```

UnionAll

SELECT UNION ALL

```
unions := uast.UnionAll(uast.NewSelect(uast.NewTable("users").As("u")).  
    Field(  
        uast.Field[string]("u", "string"),  
    ),  
)
```

Output MariaDB:

```
UNION ALL SELECT `u`.`string` FROM `users` AS `u`
```

Output MsSQL:

```
UNION ALL SELECT [u].[string] FROM [users] AS [u]
```

Output MySQL:

```
UNION ALL SELECT `u`.`string` FROM `users` AS `u`
```

Output PostgreSQL:

```
UNION ALL SELECT "u"."string" FROM "users" AS "u"
```

Output SQLite:

```
UNION ALL SELECT "u"."string" FROM "users" AS "u"
```

UnionExcept

SELECT

EXCEPT

```
unions := uast.UnionExcept(uast.NewSelect(uast.NewTable("users").As("u")).  
    Field(  
        uast.Field[string]("u", "string"),  
    ),  
)
```

Output MariaDB:

```
EXCEPT SELECT `u`.`string` FROM `users` AS `u`
```

Output MsSQL:

```
EXCEPT SELECT [u].[string] FROM [users] AS [u]
```

Output MySQL:

```
EXCEPT SELECT `u`.`string` FROM `users` AS `u`
```

Output PostgreSQL:

```
EXCEPT SELECT "u"."string" FROM "users" AS "u"
```

Output SQLite:

```
EXCEPT SELECT "u"."string" FROM "users" AS "u"
```

UnionIntersect

SELECT

INTERSECT

```
unions := uast.UnionIntersect(uast.NewSelect(uast.NewTable("users").As("u")).  
    Field(  
        uast.Field[string]("u", "string"),  
    ),  
)
```

Output MariaDB:

```
INTERSECT SELECT `u`.`string` FROM `users` AS `u`
```

Output MsSQL:

```
INTERSECT SELECT [u].[string] FROM [users] AS [u]
```

Output MySQL:

```
INTERSECT SELECT `u`.`string` FROM `users` AS `u`
```

Output PostgreSQL:

```
INTERSECT SELECT "u"."string" FROM "users" AS "u"
```

Output SQLite:

```
INTERSECT SELECT "u"."string" FROM "users" AS "u"
```

clauseValues

Pair

Pair

```
values := Values(  
    uast.Pair(uast.Field[string]("u", "string"), uast.Value("ivan")),  
    uast.Pair(uast.Field[int]("u", "number"), uast.Value(2)),  
)
```

Output MariaDB:

```
VALUES (?, ?)
```

Output MsSQL:

```
VALUES (@p1, @p2)
```

Output MySQL:

```
VALUES (?, ?)
```

Output PostgreSQL:

```
VALUES ($1, $2)
```

Output SQLite:

```
VALUES (?, ?)
```

Upsert

`Upsert` INSERT ... VALUES upsert

```
values := Values(  
    uast.Pair(uast.Field[string]("u", "string"), uast.Value("ivan")),  
    uast.Pair(uast.Field[int]("u", "number"), uast.Value(2)),  
).  
Upsert(  
    uast.Pair(uast.Field[string]("u", "string"), uast.Value("updated")),  
)
```

Output MariaDB:

```
VALUES (?, ?) ON DUPLICATE KEY UPDATE `string` = ?
```

Output MsSQL:

```
// Not supported
```

Output MySQL:

```
VALUES (?, ?) ON DUPLICATE KEY UPDATE `string` = ?
```

Output PostgreSQL:

```
VALUES ($1, $2) ON CONFLICT DO UPDATE SET "string" = $3
```

Output SQLite:

```
VALUES (?, ?) ON CONFLICT DO UPDATE SET "string" = ?
```

clauseWhere

WHERE

```
where = Where(  
    uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),  
)
```

Output MariaDB:

```
WHERE `u`.`string` = ?
```

Output MsSQL:

```
WHERE [u].[string] = @p1
```

Output MySQL:

```
WHERE `u`.`string` = ?
```

Output PostgreSQL:

```
WHERE "u"."string" = $1
```

Output SQLite:

```
WHERE "u"."string" = ?
```

clauseWith

Norecursive

WithN

CTE

```
with := WithN("cte_norecursive", NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),
    ),
    "id", "string",
)
```

Output MariaDB:

```
WITH `cte_norecursive` (`id`, `string`) AS (SELECT `u`.`id`, `u`.`string` FROM
```

Output MsSQL:

```
WITH [cte_norecursive] ([id], [string]) AS (SELECT [u].[id], [u].[string] FROM
```

Output MySQL:

```
WITH `cte_norecursive` (`id`, `string`) AS (SELECT `u`.`id`, `u`.`string` FROM
```

Output PostgreSQL:

```
WITH "cte_norecursive" ("id", "string") AS (SELECT "u"."id", "u"."string" FROM
```

Output SQLite:

```
WITH "cte_norecursive" ("id", "string") AS (SELECT "u"."id", "u"."string" FROM
```

Recursive

WithR

CTE

UnionAll

Unions

```
with := WithR("cte_recursive", NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),
    ).
    Unions(
        uast.UnionAll(uast.NewSelect(uast.NewTable("users").As("u")).
            Field(
                uast.Field[int64]("u", "id"),
                uast.Field[string]("u", "string"),
            ).
            Join(
                uast.Inner(uast.NewCTE("cte_recursive", "rec"), uast.Equal(uas
            ),
        ),
    ),
    "id", "string",
)
```

Output MariaDB:

```
WITH RECURSIVE `cte_recursive` (`id`, `string`) AS (SELECT `u`.`id`, `u`.`stri
```

Output MsSQL:

```
WITH RECURSIVE [cte_recursive] ([id], [string]) AS (SELECT [u].[id], [u].[stri
```

Output MySQL:

```
WITH RECURSIVE `cte_recursive` (`id`, `string`) AS (SELECT `u`.`id`, `u`.`stri
```

Output PostgreSQL:

```
WITH RECURSIVE "cte_recursive" ("id", "string") AS (SELECT "u"."id", "u"."stri
```

Output SQLite:

```
WITH RECURSIVE "cte_recursive" ("id", "string") AS (SELECT "u"."id", "u"."stri
```

exprArray

Array

SQL

```
array := uast.Array(0, 1, 2)
```

Output MariaDB:

```
ARRAY[?, ?, ?]
```

Output MsSQL:

```
ARRAY[@p1, @p2, @p3]
```

Output MySQL:

```
ARRAY[?, ?, ?]
```

Output PostgreSQL:

```
ARRAY[$1, $2, $3]
```

Output SQLite:

```
ARRAY[?, ?, ?]
```

exprBinary

BitwiseAnd

```
binary := uast.BitwiseAnd(uast.Field[int]("u", "number"), uast.Value(0b0010))
```

Output MariaDB:

```
`u`.`number` & ?
```

Output MsSQL:

```
[u].[number] & @p1
```

Output MySQL:

```
`u`.`number` & ?
```

Output PostgreSQL:

```
"u"."number" & $1
```

Output SQLite:

```
"u"."number" & ?
```

BitwiseOr

```
binary := uast.BitwiseOr(uast.Field[int]("u", "number"), uast.Value(0b0010))
```

Output MariaDB:

```
`u`.`number` | ?
```

Output MsSQL:

```
[u].[number] | @p1
```

Output MySQL:

```
`u`.`number` | ?
```

Output PostgreSQL:

```
"u"."number" | $1
```

Output SQLite:

```
"u"."number" | ?
```

BitwiseXor

```
binary := uast.BitwiseXor(uast.Field[int]("u", "number"), uast.Value(0b0010))
```

Output MariaDB:

```
`u`.`number` ^ ?
```

Output MsSQL:

```
[u].[number] ^ @p1
```

Output MySQL:

```
`u`.`number` ^ ?
```

Output PostgreSQL:

```
"u"."number" ^ $1
```

Output SQLite:

```
"u"."number" ^ ?
```

Divide

```
binary := uast.Divide(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` / ?
```

Output MsSQL:

```
[u].[number] / @p1
```

Output MySQL:

```
`u`.`number` / ?
```

Output PostgreSQL:

```
"u"."number" / $1
```

Output SQLite:

```
"u"."number" / ?
```

Minus

```
binary := uast.Minus(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` - ?
```

Output MsSQL:

```
[u].[number] - @p1
```

Output MySQL:

```
`u`.`number` - ?
```

Output PostgreSQL:

```
"u"."number" - $1
```

Output SQLite:

```
"u"."number" - ?
```

Modulo

```
binary := uast.Modulo(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` % ?
```

Output MsSQL:

```
[u].[number] % @p1
```

Output MySQL:

```
`u`.`number` % ?
```

Output PostgreSQL:

```
"u"."number" % $1
```

Output SQLite:

```
"u"."number" % ?
```

Multiply

```
binary := uast.Multiply(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` * ?
```

Output MsSQL:

```
[u].[number] * @p1
```

Output MySQL:

```
`u`.`number` * ?
```

Output PostgreSQL:

```
"u"."number" * $1
```

Output SQLite:

```
"u"."number" * ?
```

Plus

```
binary := uast.Plus(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` + ?
```

Output MsSQL:

```
[u].[number] + @p1
```

Output MySQL:

```
`u`.`number` + ?
```

Output PostgreSQL:

```
"u"."number" + $1
```

Output SQLite:

```
"u"."number" + ?
```

ShiftLeft

```
binary := uast.ShiftLeft(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` << ?
```

Output MsSQL:

```
[u].[number] << @p1
```

Output MySQL:

```
`u`.`number` << ?
```

Output PostgreSQL:

```
"u"."number" << $1
```

Output SQLite:

```
"u"."number" << ?
```

ShiftRight

```
binary := uast.ShiftRight(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` >> ?
```

Output MsSQL:

```
[u].[number] >> @p1
```

Output MySQL:

```
`u`.`number` >> ?
```

Output PostgreSQL:

```
"u"."number" >> $1
```

Output SQLite:

```
"u"."number" >> ?
```

exprComparison

Between

`valueStart` `valueEnd`

```
comparison := uast.Between(uast.Field[int]("u", "number"), uast.Value(0), uast
```

Output MariaDB:

```
`u`.`number` BETWEEN ? AND ?
```

Output MsSQL:

```
[u].[number] BETWEEN @p1 AND @p2
```

Output MySQL:

```
`u`.`number` BETWEEN ? AND ?
```

Output PostgreSQL:

```
"u"."number" BETWEEN $1 AND $2
```

Output SQLite:

```
"u"."number" BETWEEN ? AND ?
```

Equal

```
=
```

```
comparison := uast.Equal(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` = ?
```

Output MsSQL:

```
[u].[number] = @p1
```

Output MySQL:

```
`u`.`number` = ?
```

Output PostgreSQL:

```
"u"."number" = $1
```

Output SQLite:

```
"u"."number" = ?
```

Exists

```
true
```

```
comparison := uast.Exists(uast.Subquery[int](uast.NewSelect(uast.ConstIntOne()
```

Output MariaDB:

```
EXISTS (SELECT 1 FROM `users` AS `u`)
```

Output MsSQL:

```
EXISTS (SELECT 1 FROM [users] AS [u])
```

Output MySQL:

```
EXISTS (SELECT 1 FROM `users` AS `u`)
```

Output PostgreSQL:

```
EXISTS (SELECT 1 FROM "users" AS "u")
```

Output SQLite:

```
EXISTS (SELECT 1 FROM "users" AS "u")
```

Greater

>

```
comparison := uast.Greater(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` > ?
```

Output MsSQL:

```
[u].[number] > @p1
```

Output MySQL:

```
`u`.`number` > ?
```

Output PostgreSQL:

```
"u"."number" > $1
```

Output SQLite:

```
"u"."number" > ?
```

GreaterEqual

>=

```
comparison := uast.GreaterEqual(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` >= ?
```

Output MsSQL:

```
[u].[number] >= @p1
```

Output MySQL:

```
`u`.`number` >= ?
```

Output PostgreSQL:

```
"u"."number" >= $1
```

Output SQLite:

```
"u"."number" >= ?
```

ILike

%

—

```
comparison := uast.ILike(uast.Field[string]("u", "string"), uast.Value("%ivan%
```

Output MariaDB:

```
LOWER(`u`.`string`) LIKE LOWER(?)
```

Output MsSQL:

`LOWER([u].[string]) LIKE LOWER(@p1)`

Output MySQL:

`LOWER(`u`.`string`) LIKE LOWER(?)`

Output PostgreSQL:

```
"u"."string" ILIKE $1
```

Output SQLite:

```
LOWER("u"."string") LIKE LOWER(?)
```

In

```
comparison := uast.In(uast.Field[string]("u", "string"), uast.Array("active",
```

Output MariaDB:

```
`u`.`string` IN (?, ?)
```

Output MsSQL:

```
[u].[string] IN (@p1, @p2)
```

Output MySQL:

```
`u`.`string` IN (?, ?)
```

Output PostgreSQL:

```
"u"."string" IN ($1, $2)
```

Output SQLite:

```
"u"."string" IN (?, ?)
```

IsNotNull

NULL

```
comparison := uast.IsNotNull(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
`u`.`string` IS NOT NULL
```

Output MsSQL:

```
[u].[string] IS NOT NULL
```

Output MySQL:

```
`u`.`string` IS NOT NULL
```

Output PostgreSQL:

```
"u"."string" IS NOT NULL
```

Output SQLite:

```
"u"."string" IS NOT NULL
```

IsNull

```
NULL
```

```
comparison := uast.IsNull(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
`u`.`string` IS NULL
```

Output MsSQL:

```
[u].[string] IS NULL
```

Output MySQL:

```
`u`.`string` IS NULL
```

Output PostgreSQL:

```
"u"."string" IS NULL
```

Output SQLite:

```
"u"."string" IS NULL
```

Less



```
comparison := uast.Less(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` < ?
```

Output MsSQL:

```
[u].[number] < @p1
```

Output MySQL:

```
`u`.`number` < ?
```

Output PostgreSQL:

```
"u"."number" < $1
```

Output SQLite:

```
"u"."number" < ?
```

LessEqual

<=

```
comparison := uast.LessEqual(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` <= ?
```

Output MsSQL:

```
[u].[number] <= @p1
```

Output MySQL:

```
`u`.`number` <= ?
```

Output PostgreSQL:

```
"u"."number" <= $1
```

Output SQLite:

```
"u"."number" <= ?
```

Like

%

_

```
comparison := uast.Like(uast.Field[string]("u", "string"), uast.Value("%ivan%"))
```

Output MariaDB:

```
`u`.`string` LIKE ?
```

Output MsSQL:

```
[u].[number] LIKE @p1
```

Output MySQL:

```
`u`.`string` LIKE ?
```

Output PostgreSQL:

```
"u"."string" LIKE $1
```

Output SQLite:

```
"u"."string" LIKE ?
```

NotBetween

`valueStart` `valueEnd`

```
comparison := uast.NotBetween(uast.Field[int]("u", "number"), uast.Value(0), uast.Value(1))
```

Output MariaDB:

```
`u`.`number` NOT BETWEEN ? AND ?
```

Output MsSQL:

```
[u].[number] NOT BETWEEN @p1 AND @p2
```

Output MySQL:

```
`u`.`number` NOT BETWEEN ? AND ?
```

Output PostgreSQL:

```
"u"."number" NOT BETWEEN $1 AND $2
```

Output SQLite:

```
"u"."number" NOT BETWEEN ? AND ?
```

NotEqual

`!=` `<>`

```
comparison := uast.NotEqual(uast.Field[int]("u", "number"), uast.Value(2))
```

Output MariaDB:

```
`u`.`number` != ?
```

Output MsSQL:

```
[u].[number] != @p1
```

Output MySQL:

```
`u`.`number` != ?
```

Output PostgreSQL:

```
"u"."number" != $1
```

Output SQLite:

```
"u"."number" != ?
```

NotExists

```
true
```

```
comparison := uast.NotExists(uast.Subquery[int](uast.NewSelect(uast.ConstIntOn
```

Output MariaDB:

```
NOT EXISTS (SELECT 1 FROM `users` AS `u`)
```

Output MsSQL:

```
NOT EXISTS (SELECT 1 FROM [users] AS [u])
```

Output MySQL:

```
NOT EXISTS (SELECT 1 FROM `users` AS `u`)
```

Output PostgreSQL:

```
NOT EXISTS (SELECT 1 FROM "users" AS "u")
```

Output SQLite:

```
NOT EXISTS (SELECT 1 FROM "users" AS "u")
```

NotILike

```
comparison := uast.NotILike(uast.Field[string]("u", "string"), uast.Value("%iv
```

Output MariaDB:

```
LOWER(`u`.`string`) NOT LIKE LOWER(?)
```

Output MsSQL:

```
LOWER([u].[string]) NOT LIKE LOWER(@p1)
```

Output MySQL:

```
LOWER(`u`.`string`) NOT LIKE LOWER(?)
```

Output PostgreSQL:

```
"u"."string" NOT ILIKE $1
```

Output SQLite:

```
LOWER("u"."string") NOT LIKE LOWER(?)
```

NotIn

```
comparison := uast.NotIn(uast.Field[string]("u", "string"), uast.Array("active"
```

Output MariaDB:

```
`u`.`string` NOT IN (?, ?)
```

Output MsSQL:

```
[u].[string] NOT IN (@p1, @p2)
```

Output MySQL:

```
`u`.`string` NOT IN (?, ?)
```

Output PostgreSQL:

```
"u"."string" NOT IN ($1, $2)
```

Output SQLite:

```
"u"."string" NOT IN (?, ?)
```

NotLike

```
comparison := uast.NotLike(uast.Field[string]("u", "string"), uast.Value("%iva
```

Output MariaDB:

```
`u`.`string` NOT LIKE ?
```

Output MsSQL:

```
[u].[string] NOT LIKE @p1
```

Output MySQL:

```
`u`.`string` NOT LIKE ?
```

Output PostgreSQL:

```
"u"."string" NOT LIKE $1
```

Output SQLite:

```
"u"."string" NOT LIKE ?
```

exprConstant

ConstBoolFalse

```
FALSE
```

```
constant := uast.ConstBoolFalse()
```

Output SQL:

```
FALSE
```

ConstBoolTrue

```
TRUE
```

```
constant := uast.ConstBoolTrue()
```

Output SQL:

```
TRUE
```

ConstFloat32One

```
1.0 float32
```

```
constant := uast.ConstFloat32One()
```

Output SQL:

```
1.0
```

ConstFloat64One

```
1.000000 float64
```

```
constant := uast.ConstFloat64One()
```

Output SQL:

```
1.000000
```

ConstIntOne

```
1 int
```

```
constant := uast.ConstIntOne()
```

Output SQL:

```
1
```

ConstInt8One

```
1 int8
```

```
constant := uast.ConstInt8One()
```

Output SQL:

```
1
```

ConstInt16One

```
1 int16
```

```
constant := uast.ConstInt160ne()
```

Output SQL:

```
1
```

ConstInt32One

1 int32

```
constant := uast.ConstInt320ne()
```

Output SQL:

```
1
```

ConstInt64One

1 int64

```
constant := uast.ConstInt640ne()
```

Output SQL:

```
1
```

ConstStringDefault

DEFAULT string

```
constant := uast.ConstStringDefault()
```

Output SQL:

```
DEFAULT
```

ConstStringNull

```
NULL string
```

```
constant := uast.ConstStringNull()
```

Output SQL:

```
NULL
```

ConstUIntOne

```
1 uint
```

```
constant := uast.ConstUIntOne()
```

Output SQL:

```
1
```

ConstUInt8One

```
1 uint8
```

```
constant := uast.ConstUInt8One()
```

Output SQL:

```
1
```

ConstUInt16One

```
1      uint16
```

```
constant := uast.ConstUInt16One()
```

Output SQL:

```
1
```

ConstUInt32One

```
1      uint32
```

```
constant := uast.ConstUInt32One()
```

Output SQL:

```
1
```

ConstUInt64One

```
1      uint64
```

```
constant := uast.ConstUint64One()
```

Output SQL:

```
1
```

exprField

Field

```
field := uast.Field[string]("u", "string")
```

Output MariaDB:

```
`u`.`string`
```

Output MsSQL:

```
[u].[string]
```

Output MySQL:

```
`u`.`string`
```

Output PostgreSQL:

```
"u"."string"
```

Output SQLite:

```
"u"."string"
```

exprFunction

Aggregate

Avg

NULL

distinct

true

```
function := uast.Avg(uast.Field[int]("u", "number"), false)
functionWithDistinct := uast.Avg(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
AVG(`u`.`number`)
AVG(DISTINCT `u`.`number`)
```

Output MsSQL:

```
AVG([u].[number])
AVG(DISTINCT [u].[number])
```

Output MySQL:

```
AVG(`u`.`number`)
AVG(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
AVG("u"."number")
AVG(DISTINCT "u"."number")
```

Output SQLite:

```
AVG("u"."number")
AVG(DISTINCT "u"."number")
```

BitAnd

```
function := uast.BitAnd(uast.Field[int]("u", "number"), false)
functionWithDistinct := uast.BitAnd(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
BIT_AND(`u`.`number`)
BIT_AND(DISTINCT `u`.`number`)
```

Output MsSQL:

```
BIT_AND([u].[number])
BIT_AND(DISTINCT [u].[number])
```

Output MySQL:

```
BIT_AND(`u`.`number`)
BIT_AND(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
BIT_AND("u"."number")  
BIT_AND(DISTINCT "u"."number")
```

Output SQLite:

```
BIT_AND("u"."number")  
BIT_AND(DISTINCT "u"."number")
```

BitOr

```
function := uast.BitOr(uast.Field[int]("u", "number"), false)  
functionWithDistinct := uast.BitOr(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
BIT_OR(`u`.`number`)  
BIT_OR(DISTINCT `u`.`number`)
```

Output MsSQL:

```
BIT_OR([u].[number])  
BIT_OR(DISTINCT [u].[number])
```

Output MySQL:

```
BIT_OR(`u`.`number`)  
BIT_OR(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
BIT_OR("u"."number")  
BIT_OR(DISTINCT "u"."number")
```

Output SQLite:

```
BIT_OR("u"."number")  
BIT_OR(DISTINCT "u"."number")
```

BitXor

```
function := uast.BitXor(uast.Field[int]("u", "number"), false)  
functionWithDistinct := uast.BitXor(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
BIT_XOR(`u`.`number`)  
BIT_XOR(DISTINCT `u`.`number`)
```

Output MsSQL:

```
BIT_XOR([u].[number])  
BIT_XOR(DISTINCT [u].[number])
```

Output MySQL:

```
BIT_XOR(`u`.`number`)  
BIT_XOR(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
BIT_XOR("u"."number")  
BIT_XOR(DISTINCT "u"."number")
```

Output SQLite:

```
BIT_XOR("u"."number")  
BIT_XOR(DISTINCT "u"."number")
```

Count

NULL

distinct

true

```
function := uast.Count(uast.Field[string]("u", "string"), false)  
functionWithDistinct := uast.Count(uast.Field[string]("u", "string"), true)
```

Output MariaDB:

```
COUNT(`u`.`string`)  
COUNT(DISTINCT `u`.`string`)
```

Output MsSQL:

```
COUNT([u].[string])  
COUNT(DISTINCT [u].[string])
```

Output MySQL:

```
COUNT(`u`.`string`)  
COUNT(DISTINCT `u`.`string`)
```

Output PostgreSQL:

```
COUNT("u"."string")  
COUNT(DISTINCT "u"."string")
```

Output SQLite:

```
COUNT("u"."string")  
COUNT(DISTINCT "u"."string")
```

GroupConcat

distinct

```
function := uast.GroupConcat(uast.Field[string]("u", "string"), false)  
functionWithDistinct := uast.GroupConcat(uast.Field[string]("u", "string"), true)
```

Output MariaDB:

```
GROUP_CONCAT(`u`.`string` SEPARATOR ',')  
GROUP_CONCAT(DISTINCT `u`.`string` SEPARATOR ',')
```

Output MsSQL:

```
GROUP_CONCAT([u].[string], ',')  
GROUP_CONCAT(DISTINCT [u].[string], ',')
```

Output MySQL:

```
GROUP_CONCAT(`u`.`string` SEPARATOR ',')  
GROUP_CONCAT(DISTINCT `u`.`string` SEPARATOR ',')
```

Output PostgreSQL:

```
STRING_AGG("u"."string", ',')
STRING_AGG(DISTINCT "u"."string", ',')
```

Output SQLite:

```
GROUP_CONCAT("u"."string" SEPARATOR ',')
GROUP_CONCAT(DISTINCT "u"."string" SEPARATOR ',')
```

Max

```
function := uast.Max(uast.Field[int]("u", "number"), false)
functionWithDistinct := uast.Max(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
MAX(`u`.`number`)
MAX(DISTINCT `u`.`number`)
```

Output MsSQL:

```
MAX([u].[number])
MAX(DISTINCT [u].[number])
```

Output MySQL:

```
MAX(`u`.`number`)
MAX(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
MAX("u"."number")
MAX(DISTINCT "u"."number")
```

Output SQLite:

```
MAX("u"."number")
MAX(DISTINCT "u"."number")
```

Min

```
function := uast.Min(uast.Field[int]("u", "number"), false)
functionWithDistinct := uast.Min(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
MIN(`u`.`number`)
MIN(DISTINCT `u`.`number`)
```

Output MsSQL:

```
MIN([u].[number])
MIN(DISTINCT [u].[number])
```

Output MySQL:

```
MIN(`u`.`number`)
MIN(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
MIN("u"."number")
MIN(DISTINCT "u"."number")
```

Output SQLite:

```
MIN("u"."number")
MIN(DISTINCT "u"."number")
```

StdDev

```
function := uast.StdDev(uast.Field[int]("u", "number"), false)
functionWithDistinct := uast.StdDev(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
STDDEV(`u`.`number`)
STDDEV(DISTINCT `u`.`number`)
```

Output MsSQL:

```
STDEV([u].[number])
STDEV(DISTINCT [u].[number])
```

Output MySQL:

```
STDDEV(`u`.`number`)
STDDEV(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
STDDEV_SAMP("u"."number")
STDDEV_SAMP(DISTINCT "u"."number")
```

Output SQLite:

```
STDEV("u"."number")
STDEV(DISTINCT "u"."number")
```

Sum

`distinct` `true`

```
function := uast.Sum(uast.Field[int]("u", "number"), false)
functionWithDistinct := uast.Sum(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
SUM(`u`.`number`)
SUM(DISTINCT `u`.`number`)
```

Output MsSQL:

```
SUM([u].[number])
SUM(DISTINCT [u].[number])
```

Output MySQL:

```
SUM(`u`.`number`)
SUM(DISTINCT `u`.`number`)
```

Output PostgreSQL:

```
SUM("u"."number")  
SUM(DISTINCT "u"."number")
```

Output SQLite:

```
SUM("u"."number")  
SUM(DISTINCT "u"."number")
```

Variance

```
function := uast.Variance(uast.Field[int]("u", "number"), false)  
functionWithDistinct := uast.Variance(uast.Field[int]("u", "number"), true)
```

Output MariaDB:

```
VARIANCE(`u`.`number`)  
VARIANCE(DISTINCT `u`.`number`)
```

Output MsSQL:

```
VAR([u].[number])  
VAR(DISTINCT [u].[number])
```

Output MySQL:

```
VARIANCE(`u`.`number`)  
VARIANCE(DISTINCT "u"."number")
```

Output PostgreSQL:

```
VAR_SAMP("u"."number")  
VAR_SAMP(DISTINCT "u"."number")
```

Output SQLite:

```
VARIANCE("u"."number")  
VARIANCE(DISTINCT "u"."number")
```

Analytical

FirstValue

OVER

```
function := uast.FirstValue(uast.Field[string]("u", "string")).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
FIRST_VALUE(`u`.`string`) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DE
```

Output MsSQL:

```
FIRST_VALUE([u].[string]) OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DE
```

Output MySQL:

```
FIRST_VALUE(`u`.`string`) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DE
```

Output PostgreSQL:

```
FIRST_VALUE("u"."string") OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
FIRST_VALUE("u"."string") OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Lag

offset

```
function := uast.Lag(uast.Field[int]("u", "number"), 2).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Asc(uast.Field[time.Time]("u", "date"))),  
)
```

Output MariaDB:

```
LAG(`u`.`number`, 2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`date` ASC)
```

Output MsSQL:

```
LAG([u].[number], 2) OVER (PARTITION BY [u].[id] ORDER BY [u].[date] ASC)
```

Output MySQL:

```
LAG(`u`.`number`, 2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`date` ASC)
```

Output PostgreSQL:

```
LAG("u"."number", 2) OVER (PARTITION BY "u"."id" ORDER BY "u"."date" ASC)
```

Output SQLite:

```
LAG("u"."number", 2) OVER (PARTITION BY "u"."id" ORDER BY "u"."date" ASC)
```

LastValue

```
function := uast.LastValue(uast.Field[string]("u", "string")).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Asc(uast.Field[int]("u", "number"))),  
    uast.RowsBetween("CURRENT ROW", "UNBOUNDED FOLLOWING"),  
)
```

Output MariaDB:

```
LAST_VALUE(`u`.`string`) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` ASC
```

Output MsSQL:

```
LAST_VALUE([u].[string]) OVER (PARTITION BY [u].[id] ORDER BY [u].[number] ASC
```

Output MySQL:

```
LAST_VALUE(`u`.`string`) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` ASC
```

Output PostgreSQL:

```
LAST_VALUE("u"."string") OVER (PARTITION BY "u"."id" ORDER BY "u"."number" ASC
```

Output SQLite:

```
LAST_VALUE("u"."string") OVER (PARTITION BY "u"."id" ORDER BY "u"."number" ASC
```

Lead

offset

```
function := uast.Lead(uast.Field[int]("u", "number"), 2).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Asc(uast.Field[time.Time]("u", "date"))),  
)
```

Output MariaDB:

```
LEAD(`u`.`number`, 2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`date` ASC)
```

Output MsSQL:

```
LEAD([u].[number], 2) OVER (PARTITION BY [u].[id] ORDER BY [u].[date] ASC)
```

Output MySQL:

```
LEAD(`u`.`number`, 2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`date` ASC)
```

Output PostgreSQL:

```
LEAD("u"."number", 2) OVER (PARTITION BY "u"."id" ORDER BY "u"."date" ASC)
```

Output SQLite:

```
LEAD("u"."number", 2) OVER (PARTITION BY "u"."id" ORDER BY "u"."date" ASC)
```

NthValue

n

```
function := uast.NthValue(uast.Field[string]("u", "string"), 2).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
    uast.RowsBetween("UNBOUNDED PRECEDING", "CURRENT ROW"),  
)
```

Output MariaDB:

```
NTH_VALUE(`u`.`string`, 2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` D
```

Output MsSQL:

```
NTH_VALUE([u].[string], 2) OVER (PARTITION BY [u].[id] ORDER BY [u].[number] D
```

Output MySQL:

```
NTH_VALUE(`u`.`string`, 2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` D
```

Output PostgreSQL:

```
NTH_VALUE("u"."string", 2) OVER (PARTITION BY "u"."id" ORDER BY "u"."number" D
```

Output SQLite:

```
NTH_VALUE("u"."string", 2) OVER (PARTITION BY "u"."id" ORDER BY "u"."number" D
```

Condition

Case

WHEN - THEN
ELSE

WHEN THEN
NULL

```
pairs := uast.CaseIf(  
    uast.CasePair(  
        uast.Less(uast.Field[int]("u", "number"), uast.Value(2)),  
        uast.Value("old"),  
    ),  
)  
elseExpr := uast.CaseElse(uast.Value("new"))  
function := uast.Case(pairs, elseExpr)
```

Output MariaDB:

```
CASE WHEN `u`.`number` < ? THEN ? ELSE ? END
```

Output MsSQL:

```
CASE WHEN [u].[number] < @p1 THEN @p2 ELSE @p3 END
```

Output MySQL:

```
CASE WHEN `u`.`number` < ? THEN ? ELSE ? END
```

Output PostgreSQL:

```
CASE WHEN "u"."number" < $1 THEN $2 ELSE $3 END
```

Output SQLite:

```
CASE WHEN "u"."number" < ? THEN ? ELSE ? END
```

Coalesce

NULL

```
function := uast.Coalesce(uast.Field[time.Time]("u", "createat"), uast.Field[t
```

Output MariaDB:

```
COALESCE(`u`.`createat`, `u`.`updateat`)
```

Output MsSQL:

```
COALESCE([u].[createat], [u].[updateat])
```

Output MySQL:

```
COALESCE(`u`.`createat`, `u`.`updateat`)
```

Output PostgreSQL:

```
COALESCE("u"."createat", "u"."updateat")
```

Output SQLite:

```
COALESCE("u"."createat", "u"."updateat")
```

Greatest

```
function := uast.Greatest(uast.Field[time.Time]("u", "createat"), uast.Field[t
```

Output MariaDB:

```
GREATEST(`u`.`createat`, `u`.`updateat`)
```

Output MsSQL:

```
GREATEST([u].[createat], [u].[updateat])
```

Output MySQL:

```
GREATEST(`u`.`createat`, `u`.`updateat`)
```

Output PostgreSQL:

```
GREATEST("u"."createat", "u"."updateat")
```

Output SQLite:

```
GREATEST("u"."createat", "u"."updateat")
```

Least

```
function := uast.Least(uast.Field[time.Time]("u", "createat"), uast.Field[time
```

Output MariaDB:

```
LEAST(`u`.`createat`, `u`.`updateat`)
```

Output MsSQL:

```
LEAST([u].[createat], [u].[updateat])
```

Output MySQL:

```
LEAST(`u`.`createat`, `u`.`updateat`)
```

Output PostgreSQL:

```
LEAST("u"."createat", "u"."updateat")
```

Output SQLite:

```
LEAST("u"."createat", "u"."updateat")
```

NullIf

NULL

```
function := uast.NullIf(uast.Field[time.Time]("u", "createat"), uast.Field[time.Time]("u", "updateat"))
```

Output MariaDB:

```
NULLIF(`u`.`createat`, `u`.`updateat`)
```

Output MsSQL:

```
NULLIF([u].[createat], [u].[updateat])
```

Output MySQL:

```
NULLIF(`u`.`createat`, `u`.`updateat`)
```

Output PostgreSQL:

```
NULLIF("u"."createat", "u"."updateat")
```

Output SQLite:

```
NULLIF("u"."createat", "u"."updateat")
```

Convert

Cast

```
function := uast.Cast(uast.Field[int]("u", "number"), uast.TypeString)
```

Output MariaDB:

```
CAST(`u`.`number` AS CHAR)
```

Output MsSQL:

```
CAST([u].[number] AS NVARCHAR)
```

Output MySQL:

```
CAST(`u`.`number` AS CHAR)
```

Output PostgreSQL:

```
CAST("u"."number" AS VARCHAR)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

CharLength

```
function := uast.CharLength(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
CHAR_LENGTH(`u`.`string`)
```

Output MsSQL:

```
CHAR_LENGTH([u].[string])
```

Output MySQL:

```
CHAR_LENGTH(`u`.`string`)
```

Output PostgreSQL:

```
CHAR_LENGTH("u"."string")
```

Output SQLite:

```
CHAR_LENGTH("u"."string")
```

DateFormat

```
function := uast.DateFormat(uast.Field[time.Time]("u", "createat"), uast.Value
```

Output MariaDB:

```
DATE_FORMAT(`u`.`createat`, '%Y-%m-%d')
```

Output MsSQL:

```
FORMAT([u].[createat], '%Y-%m-%d')
```

Output MySQL:

```
DATE_FORMAT(`u`.`createat`, '%Y-%m-%d')
```

Output PostgreSQL:

```
TO_CHAR("u"."createat", '%Y-%m-%d')
```

Output SQLite:

```
STRFTIME("u"."createat", '%Y-%m-%d')
```

Degrees

```
function := uast.Degrees(uast.Field[int]("u", "number"))
```

Output MariaDB:

```
DEGREES(`u`.`number`)
```

Output MsSQL:

```
DEGREES([u].[number])
```

Output MySQL:

```
DEGREES(`u`.`number`)
```

Output PostgreSQL:

```
DEGREES("u"."number")
```

Output SQLite:

```
DEGREES("u"."number")
```

Length

```
function := uast.Length(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
LENGTH(`u`.`string`)
```

Output MsSQL:

```
LEN([u].[string])
```

Output MySQL:

```
LENGTH(`u`.`string`)
```

Output PostgreSQL:

```
LENGTH("u"."string")
```

Output SQLite:

```
LENGTH("u"."string")
```

Position

```
function := uast.Position(uast.Field[string]("u", "string"), uast.Value("old"))
```

Output MariaDB:

```
POSITION(? IN `u`.`string`)
```

Output MsSQL:

```
CHARINDEX(@p1, [u].[string])
```

Output MySQL:

```
POSITION(? IN `u`.`string`)
```

Output PostgreSQL:

```
POSITION($1 IN "u"."string")
```

Output SQLite:

```
POSITION(? IN "u"."string")
```

Radians

```
function := uast.Radians(uast.Field[int]("u", "number"))
```

Output MariaDB:

```
RADIANS(`u`.`number`)
```

Output MsSQL:

```
RADIANS([u].[number])
```

Output MySQL:

```
RADIANS(`u`.`number`)
```

Output PostgreSQL:

```
RADIANS("u"."number")
```

Output SQLite:

```
RADIANS("u"."number")
```

Date and time

CurDate

```
function := uast.CurDate()
```

Output MariaDB:

```
CURDATE()
```

Output MsSQL:

```
CAST(GETDATE() AS DATE)
```

Output MySQL:

```
CURDATE()
```

Output PostgreSQL:

```
CURRENT_DATE
```

Output SQLite:

```
DATE('now')
```

CurTime

```
function := uast.CurTime()
```

Output MariaDB:

```
CURTIME()
```

Output MsSQL:

```
CAST(GETDATE() AS TIME)
```

Output MySQL:

```
CURTIME()
```

Output PostgreSQL:

```
CURRENT_TIME
```

Output SQLite:

```
TIME('now')
```

DateAdd

/

```
function := uast.DateAdd(uast.Field[time.Time]("u", "createat"), uast.Value("2
```

Output MariaDB:

```
DATE_ADD(`u`.`createat`, INTERVAL 2 DAY)
```

Output MsSQL:

```
DATEADD(DAY, 2, [u].[createat])
```

Output MySQL:

```
DATE_ADD(`u`.`createat`, INTERVAL 2 DAY)
```

Output PostgreSQL:

```
("u"."createat" + INTERVAL '2 DAY')
```

Output SQLite:

```
DATETIME("u"."createat", '+2 DAY')
```

DateDiff

`datetimeEnd` - `datetimeStart`

```
function := uast.DateDiff(uast.Field[time.Time]("u", "updateat"), uast.Field[t
```

Output MariaDB:

```
DATEDIFF(`u`.`updateat`, `u`.`createat`)
```

Output MsSQL:

```
DATEDIFF([u].[updateat], [u].[createat])
```

Output MySQL:

```
DATEDIFF(`u`.`updateat`, `u`.`createat`)
```

Output PostgreSQL:

```
DATE_PART('day', "u"."updateat" - "u"."createat")
```

Output SQLite:

```
DATEDIFF("u"."updateat", "u"."createat")
```

DateSub

/

```
function := uast.DateSub(uast.Field[time.Time]("u", "createat"), uast.Value("2
```

Output MariaDB:

```
DATE_SUB(`u`.`createat`, INTERVAL 2 DAY)
```

Output MsSQL:

```
DATEADD(DAY, -2, [u].[createat])
```

Output MySQL:

```
DATE_SUB(`u`.`createat`, INTERVAL 2 DAY)
```

Output PostgreSQL:

```
("u"."createat" - INTERVAL '2 DAY')
```

Output SQLite:

```
DATETIME("u"."createat", '-2 DAY')
```

Day

1-31

```
function := uast.Day(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
DAY(`u`.`createat`)
```

Output MsSQL:

```
DAY([u].[createat])
```

Output MySQL:

```
DAY(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(DAY FROM "u"."createat")
```

Output SQLite:

```
DAY("u"."createat")
```

DayName

'Monday', 'Tuesday'

```
function := uast.DayName(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
DAYNAME(`u`.`createat`)
```

Output MsSQL:

```
DATENAME(WEEKDAY, [u].[createat])
```

Output MySQL:

```
DAYNAME(`u`.`createat`)
```

Output PostgreSQL:

```
TO_CHAR("u"."createat", 'Day')
```

Output SQLite:

```
STRFTIME('%w', "u"."createat")
```

Hour

0–23

```
function := uast.Hour(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
HOUR(`u`.`createat`)
```

Output MsSQL:

```
DATEPART(HOUR, [u].[createat])
```

Output MySQL:

```
HOUR(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(HOUR FROM "u"."createat")
```

Output SQLite:

```
HOUR("u"."createat")
```

Minute

0–59

```
function := uast.Minute(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
MINUTE(`u`.`createat`)
```

Output MsSQL:

```
DATEPART(MINUTE, [u].[createat])
```

Output MySQL:

```
MINUTE(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(MINUTE FROM "u"."createat")
```

Output SQLite:

```
MINUTE("u"."createat")
```

Month

1–12

```
function := uast.Month(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
MONTH(`u`.`createat`)
```

Output MsSQL:

```
MONTH([u].[createat])
```

Output MySQL:

```
MONTH(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(MONTH FROM "u"."createat")
```

Output SQLite:

```
MONTH("u"."createat")
```

MonthName

'January', 'February'

```
function := uast.MonthName(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
MONTHNAME(`u`.`createat`)
```

Output MsSQL:

```
DATENAME(MONTH, [u].[createat])
```

Output MySQL:

```
MONTHNAME(`u`.`createat`)
```

Output PostgreSQL:

```
TO_CHAR("u"."createat", 'Month')
```

Output SQLite:

```
STRFTIME('%m', "u"."createat")
```

Now

```
function := uast.Now()
```

Output MariaDB:

```
NOW()
```

Output MsSQL:

```
GETDATE()
```

Output MySQL:

```
NOW()
```

Output PostgreSQL:

```
CURRENT_TIMESTAMP
```

Output SQLite:

```
DATETIME('now')
```

Quarter

1–4

```
function := uast.Quarter(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
QUARTER(`u`.`createat`)
```

Output MsSQL:

```
DATEPART(QUARTER, [u].[createat])
```

Output MySQL:

```
QUARTER(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(QUARTER FROM "u"."createat")
```

Output SQLite:

```
QUARTER("u"."createat")
```

Second

0–59

```
function := uast.Second(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
SECOND(`u`.`createat`)
```

Output MsSQL:

```
DATEPART(SECOND, [u].[createat])
```

Output MySQL:

```
SECOND(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(SECOND FROM "u"."createat")
```

Output SQLite:

```
SECOND("u"."createat")
```

TimeAdd

/

```
function := uast.TimeAdd(uast.Field[time.Time]("u", "createat"), uast.Value("2
```

Output MariaDB:

```
TIME_ADD(`u`.`createat`, '2 HOUR')
```

Output MsSQL:

```
DATEADD(HOUR, 2, [u].[createat])
```

Output MySQL:

```
TIME_ADD(`u`.`createat`, '2 HOUR')
```

Output PostgreSQL:

```
("u"."createat" + INTERVAL '2 HOUR')
```

Output SQLite:

```
TIME("u"."createat", '+2 HOUR')
```

TimeDiff

/ `timeEnd` - `timeStart`

```
function := uast.TimeDiff(uast.Field[time.Time]("u", "updateat"), uast.Field[t
```

Output MariaDB:

```
TIMEDIFF(`u`.`updateat`, `u`.`createat`)
```

Output MsSQL:

```
TIMEDIFF([u].[updateat], [u].[createat])
```

Output MySQL:

```
TIMEDIFF(`u`.`updateat`, `u`.`createat`)
```

Output PostgreSQL:

```
DATE_PART('time', "u"."updateat" - "u"."createat")
```

Output SQLite:

```
TIMEDIFF("u"."updateat", "u"."createat")
```

TimeSub

/

```
function := uast.TimeSub(uast.Field[time.Time]("u", "createat"), uast.Value("2
```

Output MariaDB:

```
TIME_SUB(`u`.`createat`, '2 HOUR')
```

Output MsSQL:

```
DATEADD(HOUR, -2, [u].[createat])
```

Output MySQL:

```
TIME_SUB(`u`.`createat`, '2 HOUR')
```

Output PostgreSQL:

```
("u"."createat" - INTERVAL '2 HOUR')
```

Output SQLite:

```
TIME("u"."createat", '-2 HOUR')
```

Week

1–53

```
function := uast.Week(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
WEEK(`u`.`createat`)
```

Output MsSQL:

```
DATEPART(WEEK, [u].[createat])
```

Output MySQL:

```
WEEK(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(WEEK FROM "u"."createat")
```

Output SQLite:

```
WEEK("u"."createat")
```

Year

```
function := uast.Year(uast.Field[time.Time]("u", "createat"))
```

Output MariaDB:

```
YEAR(`u`.`createat`)
```

Output MsSQL:

```
YEAR([u].[createat])
```

Output MySQL:

```
YEAR(`u`.`createat`)
```

Output PostgreSQL:

```
EXTRACT(YEAR FROM "u"."createat")
```

Output SQLite:

```
YEAR("u"."createat")
```

Json

JsonArray

JSON

```
function := uast.JsonArray(  
  uast.Field[string]("u", "json"),  
  uast.Value("val1"),  
  uast.Value("val2"),  
)
```

Output MariaDB:

```
JSON_ARRAY(`u`.`json`, ?, ?)
```

Output MsSQL:

```
JSON_ARRAY([u].[json], @p1, @p2)
```

Output MySQL:

```
JSON_ARRAY(`u`.`json`, ?, ?)
```

Output PostgreSQL:

```
JSON_ARRAY("u"."json", $1, $2)
```

Output SQLite:

```
JSON_ARRAY("u"."json", ?, ?)
```

JsonArrayAgg

JSON

```
function := uast.JsonArrayAgg(  
    uast.Field[string]("u", "json"),  
)
```

Output MariaDB:

```
JSON_ARRAYAGG(`u`.`json`)
```

Output MsSQL:

```
JSON_ARRAYAGG([u].[json])
```

Output MySQL:

```
JSON_ARRAYAGG(`u`.`json`)
```

Output PostgreSQL:

```
JSON_AGG("u"."json")
```

Output SQLite:

```
JSON_GROUP_ARRAY("u"."json")
```

JsonContains

JSON

```
function := uast.JsonContains(  
    uast.Field[string]("u", "json"),  
    uast.Value(`{"key":"val"}`),  
)
```

Output MariaDB:

```
JSON_CONTAINS(`u`.`json`, '{"key":"val"}')
```

Output MsSQL:

```
// Not supported
```

Output MySQL:

```
JSON_CONTAINS(`u`.`json`, '{"key":"val"}')
```

Output PostgreSQL:

```
("u"."json" @> '{"key":"val"}')
```

Output SQLite:

```
JSON_CONTAINS("u"."json", '{"key":"val"}')
```

JsonExtract

JSON

json

JsonPath

JsonKey / JsonIndex

```
function := JsonExtract(
  uast.Field[string]("u", "json"),
  uast.JsonGroup(
    uast.JsonPath(
      uast.JsonKey("parent"),
      uast.JsonIndex(0),
      uast.JsonKey("child"),
    ),
  ),
  uast.TypeString,
)
```

Output MariaDB:

```
(`u`.`json` ->> '$.parent[0].child')
```

Output MsSQL:

```
JSON_VALUE([u].[json], '$.parent[0].child')
```

Output MySQL:

```
(`u`.`json` ->> '$.parent[0].child')
```

Output PostgreSQL:

```
("u"."json" #>> '{parent,0,child}')
```

Output SQLite:

```
("u"."json" ->> '$.parent[0].child')
```

JsonObject

JSON

```
function := uast.JsonObject(  
  uast.JsonPair(  
    uast.JsonKey("key"),  
    uast.Count(uast.Field[string]("u", "json"), false),  
  ),  
)
```

Output MariaDB:

```
JSON_OBJECT('key', COUNT(`u`.`json`))
```

Output MsSQL:

```
JSON_OBJECT('key', COUNT([u].[json]))
```

Output MySQL:

```
JSON_OBJECT('key', COUNT(`u`.`json`))
```

Output PostgreSQL:

```
JSON_BUILD_OBJECT('key', COUNT("u"."json"))
```

Output SQLite:

```
JSON_OBJECT('key', COUNT("u"."json"))
```

JsonObjectAgg

JSON

```
function := uast.JsonObjectAgg(  
  uast.Field[string]("u", "json"),  
  uast.Field[int]("u", "number"),  
)
```

Output MariaDB:

```
JSON_OBJECTAGG(`u`.`json`, `u`.`number`)
```

Output MsSQL:

```
JSON_OBJECTAGG([u].[json], [u].[number])
```

Output MySQL:

```
JSON_OBJECTAGG(`u`.`json`, `u`.`number`)
```

Output PostgreSQL:

```
JSON_OBJECT_AGG("u"."json", "u"."number")
```

Output SQLite:

```
JSON_GROUP_OBJECT("u"."json", "u"."number")
```

JsonRemove

JSON

```
function := uast.JsonRemove(
  uast.Field[string]("u", "json"),
  uast.JsonGroup(
    uast.JsonPath(
      uast.JsonKey("key1"),
    ),
  ),
  uast.JsonGroup(
    uast.JsonPath(
      uast.JsonKey("key2"),
    ),
  ),
)
```

Output MariaDB:

```
JSON_REMOVE(`u`.`json`, '$.key1', '$.key2')
```

Output MsSQL:

```
JSON_MODIFY(JSON_MODIFY([u].[json], '$.key1', NULL), '$.key2', NULL)
```

Output MySQL:

```
JSON_REMOVE(`u`.`json`, '$.key1', '$.key2')
```

Output PostgreSQL:

```
("u"."json" - '{key1}' - '{key2}')
```

Output SQLite:

```
JSON_REMOVE("u"."json", '$.key1', '$.key2')
```

JsonSet

JSON

```
function := uast.JsonSet(  
  uast.Field[string]("u", "json"),  
  uast.JsonGroup(  
    uast.JsonPath(  
      uast.JsonKey("key1"),  
    ),  
    uast.Value("val1"),  
  ),  
  uast.JsonGroup(  
    uast.JsonPath(  
      uast.JsonKey("key2"),  
    ),  
    uast.Value("val2"),  
  ),  
)
```

Output MariaDB:

```
JSON_SET(`u`.`json`, '$.key1', ?, '$.key2', ?)
```

Output MsSQL:

```
JSON_MODIFY(JSON_MODIFY([u].[json], '$.key1', @p1), '$.key2', @p2)
```

Output MySQL:

```
JSON_SET(`u`.`json`, '$.key1', ?, '$.key2', ?)
```

Output PostgreSQL:

```
jsonb_set(jsonb_set("u"."json", '{key1}', $1), '{key2}', $2)
```

Output SQLite:

```
JSON_SET("u"."json", '$.key1', ?, '$.key2', ?)
```

JsonType

JSON 'OBJECT', 'ARRAY', 'STRING', 'INTEGER', 'NULL'

```
function := uast.JsonType(uast.Field[string]("u", "json"))
```

Output MariaDB:

```
JSON_TYPE(`u`.`json`)
```

Output MsSQL:

```
// Not supported
```

Output MySQL:

```
JSON_TYPE(`u`.`json`)
```

Output PostgreSQL:

```
jsonb_typeof("u"."json")
```

Output SQLite:

```
JSON_TYPE("u"."json")
```

Math

Abs

```
function := uast.Abs(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
ABS(`u`.`x`)
```

Output MsSQL:

```
ABS([u].[x])
```

Output MySQL:

```
ABS(`u`.`x`)
```

Output PostgreSQL:

```
ABS("u"."x")
```

Output SQLite:

```
ABS("u"."x")
```

ACos

```
function := uast.ACos(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
ACOS(`u`.`x`)
```

Output MsSQL:

```
ACOS([u].[x])
```

Output MySQL:

```
ACOS(`u`.`x`)
```

Output PostgreSQL:

```
ACOS("u"."x")
```

Output SQLite:

```
ACOS("u"."x")
```

ASin

```
function := uast.ASin(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
ASIN(`u`.`x`)
```

Output MsSQL:

```
ASIN([u].[x])
```

Output MySQL:

```
ASIN(`u`.`x`)
```

Output PostgreSQL:

```
ASIN("u"."x")
```

Output SQLite:

```
ASIN("u"."x")
```

ATan

```
function := uast.ATan(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
ATAN(`u`.`x`)
```

Output MsSQL:

```
ATAN([u].[x])
```

Output MySQL:

```
ATAN(`u`.`x`)
```

Output PostgreSQL:

```
ATAN("u"."x")
```

Output SQLite:

```
ATAN("u"."x")
```

ATan2

y / x

```
function := uast.ATan2(uast.Field[int]("u", "y"), uast.Field[int]("u", "x"))
```

Output MariaDB:

```
ATAN2(`u`.`y`, `u`.`x`)
```

Output MsSQL:

```
ATAN2([u].[y], [u].[x])
```

Output MySQL:

```
ATAN2(`u`.`y`, `u`.`x`)
```

Output PostgreSQL:

```
ATAN2("u"."y", "u"."x")
```

Output SQLite:

```
ATAN2("u"."y", "u"."x")
```

Cbrt

```
function := uast.Cbrt(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
CBRT(`u`.`x`)
```

Output MsSQL:

```
CBRT([u].[x])
```

Output MySQL:

```
CBRT(`u`.`x`)
```

Output PostgreSQL:

```
CBRT("u"."x")
```

Output SQLite:

```
CBRT("u"."x")
```

Ceil

```
function := uast.Ceil(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
CEILING(`u`.`x`)
```

Output MsSQL:

```
CEILING([u].[x])
```

Output MySQL:

```
CEILING(`u`.`x`)
```

Output PostgreSQL:

```
CEIL("u"."x")
```

Output SQLite:

```
CEIL("u"."x")
```

Cos

```
function := uast.Cos(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
COS(`u`.`x`)
```

Output MsSQL:

```
COS([u].[x])
```

Output MySQL:

```
COS(`u`.`x`)
```

Output PostgreSQL:

```
COS("u"."x")
```

Output SQLite:

```
COS("u"."x")
```

Exp

e

~2.71828

```
function := uast.Exp(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
EXP(`u`.`x`)
```

Output MsSQL:

```
EXP([u].[x])
```

Output MySQL:

```
EXP(`u`.`x`)
```

Output PostgreSQL:

```
EXP("u"."x")
```

Output SQLite:

```
EXP("u"."x")
```

Floor

```
function := uast.Floor(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
FLOOR(`u`.`x`)
```

Output MsSQL:

```
FLOOR([u].[x])
```

Output MySQL:

```
FLOOR(`u`.`x`)
```

Output PostgreSQL:

```
FLOOR("u"."x")
```

Output SQLite:

```
FLOOR("u"."x")
```

Ln

e

```
function := uast.Ln(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
LN(`u`.`x`)
```

Output MsSQL:

```
LN([u].[x])
```

Output MySQL:

```
LN(`u`.`x`)
```

Output PostgreSQL:

```
LN("u"."x")
```

Output SQLite:

```
LN("u"."x")
```

Log

```
function := uast.Log(uast.Field[int]("u", "x"), uast.Value(2))
```

Output MariaDB:

```
LOG(`u`.`x`, ?)
```

Output MsSQL:

```
LOG([u].[x], @p1)
```

Output MySQL:

```
LOG(`u`.`x`, ?)
```

Output PostgreSQL:

```
LOG("u"."x", $1)
```

Output SQLite:

```
LOG("u"."x", ?)
```

Mod

```
function := uast.Mod(uast.Field[int]("u", "x"), uast.Value(2))
```

Output MariaDB:

```
MOD(`u`.`x`, ?)
```

Output MsSQL:

```
MOD([u].[x], @p1)
```

Output MySQL:

```
MOD(`u`.`x`, ?)
```

Output PostgreSQL:

```
MOD("u"."x", $1)
```

Output SQLite:

```
MOD("u"."x", ?)
```

Pi

π ~3.14159

```
function := uast.Pi()
```

Output MariaDB:

```
PI()
```

Output MsSQL:

```
PI()
```

Output MySQL:

```
PI()
```

Output PostgreSQL:

```
PI()
```

Output SQLite:

```
PI()
```

Power

exponent

```
function := uast.Power(uast.Field[int]("u", "x"), uast.Value(2))
```

Output MariaDB:

```
POWER(`u`.`x`, ?)
```

Output MsSQL:

```
POWER([u].[x], @p1)
```

Output MySQL:

```
POWER(`u`.`x`, ?)
```

Output PostgreSQL:

```
POWER("u"."x", $1)
```

Output SQLite:

```
POWER("u"."x", ?)
```

Rand

0,1

```
function := uast.Rand()
```

Output MariaDB:

```
RAND()
```

Output MsSQL:

```
RAND()
```

Output MySQL:

```
RAND()
```

Output PostgreSQL:

```
RANDOM()
```

Output SQLite:

```
RANDOM()
```

Round

```
function := uast.Round(uast.Field[int]("u", "x"), uast.Value(2))
```

Output MariaDB:

```
ROUND(`u`.`x`, ?)
```

Output MsSQL:

```
ROUND([u].[x], @p1)
```

Output MySQL:

```
ROUND(`u`.`x`, ?)
```

Output PostgreSQL:

```
ROUND("u"."x", $1)
```

Output SQLite:

```
ROUND("u"."x", ?)
```

Sin

```
function := uast.Sin(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
SIN(`u`.`x`)
```

Output MsSQL:

```
SIN([u].[x])
```

Output MySQL:

```
SIN(`u`.`x`)
```

Output PostgreSQL:

```
SIN("u"."x")
```

Output SQLite:

```
SIN("u"."x")
```

Sqrt

```
function := uast.Sqrt(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
SQRT(`u`.`x`)
```

Output MsSQL:

```
SQRT([u].[x])
```

Output MySQL:

```
SQRT(`u`.`x`)
```

Output PostgreSQL:

```
SQRT("u"."x")
```

Output SQLite:

```
SQRT("u"."x")
```

Tan

```
function := uast.Tan(uast.Field[int]("u", "x"))
```

Output MariaDB:

```
TAN(`u`.`x`)
```

Output MsSQL:

```
TAN([u].[x])
```

Output MySQL:

```
TAN(`u`.`x`)
```

Output PostgreSQL:

```
TAN("u"."x")
```

Output SQLite:

```
TAN("u"."x")
```

Trunc

```
function := uast.Trunc(uast.Field[int]("u", "x"), uast.Value(2))
```

Output MariaDB:

```
TRUNCATE(`u`.`x`, ?)
```

Output MsSQL:

```
ROUND([u].[x], @p1, 1)
```

Output MySQL:

```
TRUNCATE(`u`.`x`, ?)
```

Output PostgreSQL:

```
TRUNC("u"."x", $1)
```

Output SQLite:

```
TRUNC("u"."x", ?)
```

Ranking

CumeDist

OVER

```
function := uast.CumeDist().Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
CUME_DIST() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
CUME_DIST() OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
CUME_DIST() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
CUME_DIST() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
CUME_DIST() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

DenseRank

OVER

```
function := uast.DenseRank().Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
DENSE_RANK() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
DENSE_RANK() OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
DENSE_RANK() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
DENSE_RANK() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
DENSE_RANK() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

NTile

n

1

n

```
function := uast.NTile(2).Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
NTILE(2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
NTILE(2) OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
NTILE(2) OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
NTILE(2) OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
NTILE(2) OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

PercentRank

0 1 0 OVER

```
function := uast.PercentRank().Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
PERCENT_RANK() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
PERCENT_RANK() OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
PERCENT_RANK() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
PERCENT_RANK() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
PERCENT_RANK() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Rank

OVER

```
function := uast.Rank().Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
RANK() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
RANK() OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
RANK() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
RANK() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
RANK() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

RowNumber

```
function := uast.RowNumber().Over(  
    uast.PartitionBy(uast.Field[int64]("u", "id")),  
    uast.OrderBy(uast.Desc(uast.Field[int]("u", "number"))),  
)
```

Output MariaDB:

```
ROW_NUMBER() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output MsSQL:

```
ROW_NUMBER() OVER (PARTITION BY [u].[id] ORDER BY [u].[number] DESC)
```

Output MySQL:

```
ROW_NUMBER() OVER (PARTITION BY `u`.`id` ORDER BY `u`.`number` DESC)
```

Output PostgreSQL:

```
ROW_NUMBER() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

Output SQLite:

```
ROW_NUMBER() OVER (PARTITION BY "u"."id" ORDER BY "u"."number" DESC)
```

String

Concat

NULL

```
function := uast.Concat(uast.Field[string]("u", "string"), uast.Value("old"),
```

Output MariaDB:

```
CONCAT(`u`.`string`, ?, ?)
```

Output MsSQL:

```
CONCAT([u].[string], @p1, @p2)
```

Output MySQL:

```
CONCAT(`u`.`string`, ?, ?)
```

Output PostgreSQL:

```
CONCAT("u"."string", $1, $2)
```

Output SQLite:

```
CONCAT("u"."string", ?, ?)
```

ConcatWs

NULL

```
function := uast.ConcatWs(uast.Value("_"), uast.Field[string]("u", "string"),
```

Output MariaDB:

```
CONCAT_WS(?, `u`.`string`, ?, ?)
```

Output MsSQL:

```
CONCAT_WS(@1, [u].[string], @2, @3)
```

Output MySQL:

```
CONCAT_WS(?, `u`.`string`, ?, ?)
```

Output PostgreSQL:

```
CONCAT_WS($1, "u"."string", $2, $3)
```

Output SQLite:

```
CONCAT_WS(?, "u"."string", ?, ?)
```

LeftString

count

```
function := uast.LeftString(uast.Field[string]("u", "string"), uast.Value(2))
```

Output MariaDB:

```
LEFT(`u`.`string`, ?)
```

Output MsSQL:

```
LEFT([u].[string], @p1)
```

Output MySQL:

```
LEFT(`u`.`string`, ?)
```

Output PostgreSQL:

```
LEFT("u"."string", $1)
```

Output SQLite:

```
LEFT("u"."string", ?)
```

Lower

```
function := uast.Lower(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
LOWER(`u`.`string`)
```

Output MsSQL:

```
LOWER([u].[string])
```

Output MySQL:

```
LOWER(`u`.`string`)
```

Output PostgreSQL:

```
LOWER("u"."string")
```

Output SQLite:

```
LOWER("u"."string")
```

LPad

count

```
function := uast.LPad(uast.Field[string]("u", "string"), uast.Value(2), uast.V
```

Output MariaDB:

```
LPAD(`u`.`string`, ?, ?)
```

Output MsSQL:

```
LPAD([u].[string], @p1, @p2)
```

Output MySQL:

```
LPAD(`u`.`string`, ?, ?)
```

Output PostgreSQL:

```
LPAD("u"."string", $1, $2)
```

Output SQLite:

```
LPAD("u"."string", ?, ?)
```

LTrim

```
function := uast.LTrim(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
LTRIM(`u`.`string`)
```

Output MsSQL:

```
LTRIM([u].[string])
```

Output MySQL:

```
LTRIM(`u`.`string`)
```

Output PostgreSQL:

```
LTRIM("u"."string")
```

Output SQLite:

```
LTRIM("u"."string")
```

Repeat

count

```
function := uast.Repeat(uast.Field[string]("u", "string"), uast.Value(2))
```

Output MariaDB:

```
REPEAT(`u`.`string`, ?)
```

Output MsSQL:

```
REPEAT([u].[string], @p1)
```

Output MySQL:

```
REPEAT(`u`.`string`, ?)
```

Output PostgreSQL:

```
REPEAT("u"."string", $1)
```

Output SQLite:

```
REPEAT("u"."string", ?)
```

Replace

```
function := uast.Replace(uast.Field[string]("u", "string"), uast.Value("old"),
```

Output MariaDB:

```
REPLACE(`u`.`string`, ?, ?)
```

Output MsSQL:

```
REPLACE([u].[string], @p1, @p2)
```

Output MySQL:

```
REPLACE(`u`.`string`, ?, ?)
```

Output PostgreSQL:

```
REPLACE("u"."string", $1, $2)
```

Output SQLite:

```
REPLACE("u"."string", ?, ?)
```

Reverse

```
function := uast.Reverse(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
REVERSE(`u`.`string`)
```

Output MsSQL:

```
REVERSE([u].[string])
```

Output MySQL:

```
REVERSE(`u`.`string`)
```

Output PostgreSQL:

```
REVERSE("u"."string")
```

Output SQLite:

```
REVERSE("u"."string")
```

RightString

count

```
function := uast.RightString(uast.Field[string]("u", "string"), uast.Value(2))
```

Output MariaDB:

```
RIGHT(`u`.`string`, ?)
```

Output MsSQL:

```
RIGHT([u].[string], @p1)
```

Output MySQL:

```
RIGHT(`u`.`string`, ?)
```

Output PostgreSQL:

```
RIGHT("u"."string", $1)
```

Output SQLite:

```
RIGHT("u"."string", ?)
```

RPad

count

```
function := uast.RPad(uast.Field[string]("u", "string"), uast.Value(2), uast.V
```

Output MariaDB:

```
RPAD(`u`.`string`, ?, ?)
```

Output MsSQL:

```
RPAD([u].[string], @p1, @p2)
```

Output MySQL:

```
RPAD(`u`.`string`, ?, ?)
```

Output PostgreSQL:

```
RPAD("u"."string", $1, $2)
```

Output SQLite:

```
RPAD("u"."string", ?, ?)
```

RTrim

```
function := uast.RTrim(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
RTRIM(`u`.`string`)
```

Output MsSQL:

```
RTRIM([u].[string])
```

Output MySQL:

```
RTRIM(`u`.`string`)
```

Output PostgreSQL:

```
RTRIM("u"."string")
```

Output SQLite:

```
RTRIM("u"."string")
```

SubString

startPos

1

lengthStr

```
function := uast.SubString(uast.Field[string]("u", "string"), uast.Value(0), u
```

Output MariaDB:

```
SUBSTRING(`u`.`string`, ?, ?)
```

Output MsSQL:

```
SUBSTRING([u].[string], @p1, @p2)
```

Output MySQL:

```
SUBSTRING(`u`.`string`, ?, ?)
```

Output PostgreSQL:

```
SUBSTRING("u"."string", $1, $2)
```

Output SQLite:

```
SUBSTRING("u"."string", ?, ?)
```

Trim

```
function := uast.Trim(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
TRIM(`u`.`string`)
```

Output MsSQL:

```
TRIM([u].[string])
```

Output MySQL:

```
TRIM(`u`.`string`)
```

Output PostgreSQL:

```
TRIM("u"."string")
```

Output SQLite:

```
TRIM("u"."string")
```

Upper

```
function := uast.Upper(uast.Field[string]("u", "string"))
```

Output MariaDB:

```
UPPER(`u`.`string`)
```

Output MsSQL:

```
UPPER([u].[string])
```

Output MySQL:

```
UPPER(`u`.`string`)
```

Output PostgreSQL:

```
UPPER("u"."string")
```

Output SQLite:

```
UPPER("u"."string")
```

exprLiteral

Literal

SQL

Value

—

```
literal := uast.Literal("%Y-%m-%d")
```

Output SQL:

```
'%Y-%m-%d'
```

exprLogical

And

AND

```
logical := uast.And(  
    uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),  
    uast.Greater(uast.Field[int]("u", "number"), uast.Value(2)),  
)
```

Output MariaDB:

```
(`u`.`string` = ? AND `u`.`number` > ?)
```

Output MsSQL:

```
([u].[string] = @p1 AND [u].[number] > @p2)
```

Output MySQL:

```
(`u`.`string` = ? AND `u`.`number` > ?)
```

Output PostgreSQL:

```
("u"."string" = $1 AND "u"."number" > $2)
```

Output SQLite:

```
("u"."string" = ? AND "u"."number" > ?)
```

Or

OR

```
logical := uast.Or(  
    uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),  
    uast.Greater(uast.Field[int]("u", "number"), uast.Value(2)),  
)
```

Output MariaDB:

```
(`u`.`string` = ? OR `u`.`number` > ?)
```

Output MsSQL:

```
([u].[string] = @p1 OR [u].[number] > @p2)
```

Output MySQL:

```
(`u`.`string` = ? OR `u`.`number` > ?)
```

Output PostgreSQL:

```
("u"."string" = $1 OR "u"."number" > $2)
```

Output SQLite:

```
("u"."string" = ? OR "u"."number" > ?)
```

exprSubquery

Subquery

SELECT

In

Exists

Equal

SELECT

u

```
subquery := uast.Subquery[int64](uast.NewSelect(uast.Field[int64]("u", "id")).
```

Output MariaDB:

```
(SELECT `u`.`id` FROM `users` AS `u`)
```

Output MsSQL:

```
(SELECT [u].[id] FROM [users] AS [u])
```

Output MySQL:

```
(SELECT `u`.`id` FROM `users` AS `u`)
```

Output PostgreSQL:

```
(SELECT "u"."id" FROM "users" AS "u")
```

Output SQLite:

```
(SELECT "u"."id" FROM "users" AS "u")
```

exprValue

Value

Go SQL — ?

\$1 Build() SQL

bool float32 float64 int int8 int16 int32

int64 uint uint8 uint16 uint32 uint64 string time.Time

```
var data string = "ivan"
value := uast.Value(data)
```

Output MariaDB:

?

Output MsSQL:

@p1

Output MySQL:

?

Output PostgreSQL:

\$1

Output SQLite:

?

API / Core /

Binary

Datetime

Numeric

String

Special

21

Cast

SQL

Binary

TypeBinary

```
binary := uast.Cast(uast.Field[int]("u", "number"), uast.TypeBinary)
```

Output MariaDB:

```
CAST(`u`.`number` AS BINARY)
```

Output MsSQL:

```
CAST([u].[number] AS BINARY)
```

Output MySQL:

```
CAST(`u`.`number` AS BINARY)
```

Output PostgreSQL:

```
CAST("u"."number" AS BYTEA)
```

Output SQLite:

```
CAST("u"."number" AS BLOB)
```

TypeVarBinary

```
binary := uast.Cast(uast.Field[int]("u", "number"), uast.TypeVarBinary)
```

Output MariaDB:

```
CAST(`u`.`number` AS VARBINARY)
```

Output MsSQL:

```
CAST([u].[number] AS VARBINARY)
```

Output MySQL:

```
CAST(`u`.`number` AS VARBINARY)
```

Output PostgreSQL:

```
CAST("u"."number" AS BYTEA)
```

Output SQLite:

```
CAST("u"."number" AS BLOB)
```

Datetime

TypeDate

```
datetime := uast.Cast(uast.Field[int]("u", "number"), uast.TypeDate)
```

Output MariaDB:

```
CAST(`u`.`number` AS DATE)
```

Output MsSQL:

```
CAST([u].[number] AS DATE)
```

Output MySQL:

```
CAST(`u`.`number` AS DATE)
```

Output PostgreSQL:

```
CAST("u"."number" AS DATE)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeDateTime

```
datetime := uast.Cast(uast.Field[int]("u", "number"), uast.TypeDateTime)
```

Output MariaDB:

```
CAST(`u`.`number` AS DATETIME)
```

Output MsSQL:

```
CAST([u].[number] AS DATETIME2)
```

Output MySQL:

```
CAST(`u`.`number` AS DATETIME)
```

Output PostgreSQL:

```
CAST("u"."number" AS TIMESTAMP)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeTime

```
datetime := uast.Cast(uast.Field[int]("u", "number"), uast.TypeTime)
```

Output MariaDB:

```
CAST(`u`.`number` AS TIME)
```

Output MsSQL:

```
CAST([u].[number] AS TIME)
```

Output MySQL:

```
CAST(`u`.`number` AS TIME)
```

Output PostgreSQL:

```
CAST("u"."number" AS TIME)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeTimestamp

```
datetime := uast.Cast(uast.Field[int]("u", "number"), uast.TypeTimestamp)
```

Output MariaDB:

```
CAST(`u`.`number` AS TIMESTAMP)
```

Output MsSQL:

```
CAST([u].[number] AS DATETIME2)
```

Output MySQL:

```
CAST(`u`.`number` AS TIMESTAMP)
```

Output PostgreSQL:

```
CAST("u"."number" AS TIMESTAMPTZ)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

Numeric

TypeBigInt

```
math := uast.Cast(uast.Field[int]("u", "number"), uast.TypeBigInt)
```

Output MariaDB:

```
CAST(`u`.`number` AS SIGNED)
```

Output MsSQL:

```
CAST([u].[number] AS BIGINT)
```

Output MySQL:

```
CAST(`u`.`number` AS SIGNED)
```

Output PostgreSQL:

```
CAST("u"."number" AS BIGINT)
```

Output SQLite:

```
CAST("u"."number" AS INTEGER)
```

TypeDecimal

```
math := uast.Cast(uast.Field[int]("u", "number"), uast.TypeDecimal)
```

Output MariaDB:

```
CAST(`u`.`number` AS DECIMAL)
```

Output MsSQL:

```
CAST([u].[number] AS DECIMAL)
```

Output MySQL:

```
CAST(`u`.`number` AS DECIMAL)
```

Output PostgreSQL:

```
CAST("u"."number" AS DECIMAL)
```

Output SQLite:

```
CAST("u"."number" AS REAL)
```

TypeDouble

```
math := uast.Cast(uast.Field[int]("u", "number"), uast.TypeDouble)
```

Output MariaDB:

```
CAST(`u`.`number` AS DECIMAL)
```

Output MsSQL:

```
CAST([u].[number] AS FLOAT)
```

Output MySQL:

```
CAST(`u`.`number` AS DECIMAL)
```

Output PostgreSQL:

```
CAST("u"."number" AS DOUBLE PRECISION)
```

Output SQLite:

```
CAST("u"."number" AS REAL)
```

TypeFloat

```
math := uast.Cast(uast.Field[int]("u", "number"), uast.TypeFloat)
```

Output MariaDB:

```
CAST(`u`.`number` AS DECIMAL)
```

Output MsSQL:

```
CAST([u].[number] AS REAL)
```

Output MySQL:

```
CAST(`u`.`number` AS DECIMAL)
```

Output PostgreSQL:

```
CAST("u"."number" AS REAL)
```

Output SQLite:

```
CAST("u"."number" AS REAL)
```

TypeInt

```
math := uast.Cast(uast.Field[int]("u", "number"), uast.TypeInt)
```

Output MariaDB:

```
CAST(`u`.`number` AS SIGNED)
```

Output MsSQL:

```
CAST([u].[number] AS INT)
```

Output MySQL:

```
CAST(`u`.`number` AS SIGNED)
```

Output PostgreSQL:

```
CAST("u"."number" AS INTEGER)
```

Output SQLite:

```
CAST("u"."number" AS INTEGER)
```

TypeSmallInt

```
math := uast.Cast(uast.Field[int]("u", "number"), uast.TypeSmallInt)
```

Output MariaDB:

```
CAST(`u`.`number` AS SIGNED)
```

Output MsSQL:

```
CAST([u].[number] AS SMALLINT)
```

Output MySQL:

```
CAST(`u`.`number` AS SIGNED)
```

Output PostgreSQL:

```
CAST("u"."number" AS SMALLINT)
```

Output SQLite:

```
CAST("u"."number" AS INTEGER)
```

String

TypeChar

```
str := uast.Cast(uast.Field[int]("u", "number"), uast.TypeChar)
```

Output MariaDB:

```
CAST(`u`.`number` AS CHAR)
```

Output MsSQL:

```
CAST([u].[number] AS CHAR)
```

Output MySQL:

```
CAST(`u`.`number` AS CHAR)
```

Output PostgreSQL:

```
CAST("u"."number" AS CHAR)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeString

```
str := uast.Cast(uast.Field[int]("u", "number"), uast.TypeString)
```

Output MariaDB:

```
CAST(`u`.`number` AS VARCHAR)
```

Output MsSQL:

```
CAST([u].[number] AS NVARCHAR)
```

Output MySQL:

```
CAST(`u`.`number` AS VARCHAR)
```

Output PostgreSQL:

```
CAST("u"."number" AS VARCHAR)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeText

```
str := uast.Cast(uast.Field[int]("u", "number"), uast.TypeText)
```

Output MariaDB:

```
CAST(`u`.`number` AS TEXT)
```

Output MsSQL:

```
CAST([u].[number] AS NVARCHAR(MAX))
```

Output MySQL:

```
CAST(`u`.`number` AS TEXT)
```

Output PostgreSQL:

```
CAST("u"."number" AS TEXT)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeVarChar

```
str := uast.Cast(uast.Field[int]("u", "number"), uast.TypeVarChar)
```

Output MariaDB:

```
CAST(`u`.`number` AS VARCHAR)
```

Output MsSQL:

```
CAST([u].[number] AS NVARCHAR)
```

Output MySQL:

```
CAST(`u`.`number` AS VARCHAR)
```

Output PostgreSQL:

```
CAST("u"."number" AS VARCHAR)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

Special

ToArray

```
special := uast.Cast(uast.Field[int]("u", "number"), uast.TypeArray)
```

Output MariaDB:

```
CAST(`u`.`number` AS JSON)
```

Output MsSQL:

```
CAST([u].[number] AS NVARCHAR(MAX))
```

Output MySQL:

```
CAST(`u`.`number` AS JSON)
```

Output PostgreSQL:

```
CAST("u"."number" AS ARRAY)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeBoolean

/

```
special := uast.Cast(uast.Field[int]("u", "number"), uast.TypeBoolean)
```

Output MariaDB:

```
CAST(`u`.`number` AS TINYINT(1))
```

Output MsSQL:

```
CAST([u].[number] AS BIT)
```

Output MySQL:

```
CAST(`u`.`number` AS TINYINT(1))
```

Output PostgreSQL:

```
CAST("u"."number" AS BOOLEAN)
```

Output SQLite:

```
CAST("u"."number" AS INTEGER)
```

TypeJSON

JSON

```
special := uast.Cast(uast.Field[int]("u", "number"), uast.TypeJSON)
```

Output MariaDB:

```
CAST(`u`.`number` AS JSON)
```

Output MsSQL:

```
CAST([u].[number] AS NVARCHAR(MAX))
```

Output MySQL:

```
CAST(`u`.`number` AS JSON)
```

Output PostgreSQL:

```
CAST("u"."number" AS JSONB)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeUUID

UUID

```
special := uast.Cast(uast.Field[int]("u", "number"), uast.TypeUUID)
```

Output MariaDB:

```
CAST(`u`.`number` AS UUID)
```

Output MsSQL:

```
CAST([u].[number] AS UNIQUEIDENTIFIER)
```

Output MySQL:

```
CAST(`u`.`number` AS CHAR(36))
```

Output PostgreSQL:

```
CAST("u"."number" AS UUID)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

TypeXML

XML

```
special := uast.Cast(uast.Field[int]("u", "number"), uast.TypeXML)
```

Output MariaDB:

```
CAST(`u`.`number` AS TEXT)
```

Output MsSQL:

```
CAST([u].[number] AS XML)
```

Output MySQL:

```
CAST(`u`.`number` AS TEXT)
```

Output PostgreSQL:

```
CAST("u"."number" AS XML)
```

Output SQLite:

```
CAST("u"."number" AS TEXT)
```

API / SQL /

NewSQL

SQL

```

builder := uast.NewSQL(
    uast.WithDialect(uast.DialectMySQL),
    uast.WithMutate(false),
)
defer builder.Close()
stmt := uast.NewSelect(uast.NewTable("users", "u")).
    Fields(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "name"),
    ).
    Where(
        uast.Equal(uast.Field[string]("u", "status"), uast.Value("active")),
    )
query1, args1, err1 := builder.Build(stmt)
if err1 != nil {
    log.Fatal(err1)
}
builder.SetDialect(uast.DialectPostgreSQL)
query2, args2, err2 := builder.Build(stmt)
if err2 != nil {
    log.Fatal(err2)
}

```

Output MySQL:

```

SELECT `u`.`id`, `u`.`name` FROM `users` AS `u` WHERE `u`.`status` = ?

```

Output PostgreSQL:

```

SELECT "u"."id", "u"."name" FROM "users" AS "u" WHERE "u"."status" = $1

```

Name	Description	Values	Default
<code>WithDialect()</code>	Sets the SQL dialect for the builder	DialectMariaDB, DialectMsSQL, DialectMySQL DialectPostgreSQL, DialectSQLite	DialectPostgreSQL
<code>WithMutate()</code>	Controls whether the builder modifies the AST in place or clones it before building	true, false	false

Name	Description	Returns
<code>Build()</code>	Compiles a statement into a SQL string and a list of arguments	<i>string</i> , [] <i>any</i> , <i>error</i>
<code>Exec()</code>	Builds and executes a statement that doesn't return rows <i>e.g.</i> , <i>INSERT</i> , <i>UPDATE</i> , <i>DELETE</i> . Uses <code>db.Exec()</code> internally	<i>sql.Result</i> , <i>error</i>
<code>Query()</code>	Builds and executes a query that returns multiple rows <i>e.g.</i> , <i>SELECT</i> . Uses <code>db.Query()</code> internally	<i>*sql.Rows</i> , <i>error</i>
<code>QueryRow()</code>	Builds and executes a query that returns at most one row. Uses <code>db.QueryRow()</code> internally	<i>*sql.Row</i> , <i>error</i>

API / SQL /

	SQL	<code>Exec</code>	<code>Query</code>	<code>QueryRow</code>
<code>Build()</code>	<code>database/sql</code>			

sqlBuilder

Build

	SQL	UAST	—	AST
SQL				

```
stmt := uast.NewSelect(uast.NewTable("users", "u")).
    Fields(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "name"),
    ).
    Where(
        uast.Equal(uast.Field[string]("u", "status"), uast.Value("active")),
    )
query, args, err := builder.Build(stmt)
```

Output SQL:

```
// Executes: SELECT "u"."id", "u"."name" FROM "users" AS "u" WHERE "u"."status"
// Returns: [active]
```

Exec

`db.Exec()` `sql.Result` INSERT
UPDATE DELETE

```
stmt := uast.NewInsert(uast.NewTable("users").As("u")).
    Values(
        uast.Pair(uast.Field[string]("u", "string"), uast.Value("ivan")),
    )
builder := uast.NewSQL(uast.WithDialect(uast.DialectPostgreSQL))
defer builder.Close()
db, _ := sql.Open("postgres", "postgres://user:pass@localhost/db")
defer db.Close()
result, err := builder.Exec(stmt, db)
if err != nil {
    log.Fatal(err)
}
rowsAffected, _ := result.RowsAffected()
```

Output SQL:

```
// Executes: INSERT INTO "users" AS "u" ("string") VALUES ($1)
// Returns: sql.Result with LastInsertId and RowsAffected
```

Query

`db.Query()``*sql.Rows``SELECT`

```
stmt := uast.NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[string]("u", "string"), uast.Value("active")),
    )
builder := uast.NewSQL(uast.WithDialect(uast.DialectPostgreSQL))
defer sbuilderql.Close()
db, _ := sql.Open("postgres", "postgres://user:pass@localhost/db")
defer db.Close()
rows, err := builder.Query(stmt, db)
if err != nil {
    log.Fatal(err)
}
defer rows.Close()
for rows.Next() {
    var id int64
    var str string
    rows.Scan(&id, &str)
    fmt.Printf("id: %d, string: %s\n", id, str)
}
```

Output SQL:

```
// Executes: SELECT "u"."id", "u"."string" FROM "users" AS "u" WHERE "u"."string" = 'active'
// Returns: *sql.Rows iterator
```

QueryRow

`db.QueryRow()``*sql.Row``Build()``SELECT`

```

stmt := uast.NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[int64]("u", "id"),
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[int64]("u", "id"), uast.Value(1)),
    )
builder := uast.NewSQL(uast.WithDialect(uast.DialectPostgreSQL))
defer builder.Close()
db, _ := sql.Open("postgres", "postgres://user:pass@localhost/db")
defer db.Close()
row, err := builder.QueryRow(stmt, db)
if err != nil {
    log.Fatal(err)
}
var id int64
var str string
err = row.Scan(&id, &str)
if err != nil {
    log.Fatal(err)
}
fmt.Printf("id: %d, string: %s\n", id, str)

```

Output SQL:

```

// Executes: SELECT "u"."id", "u"."string" FROM "users" AS "u" WHERE "u"."id" = 1
// Returns: *sql.Row, scanned via row.Scan()

```

Build() database/sql

Build() *sql.DB *sql.Tx QueryRow *sql.Row —

Scan() Build()

API / SQL /

SQL

WithDialect

WithMutate

SetDialect

SetMutate

WithDialect/SetDialect

WithDialect

SetDialect

```

stmt := uast.NewSelect(uast.Field[string]("u", "string")).
    From(
        uast.NewTable("users").As("u"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "id"), uast.Value(1)),
    )
builder := uast.NewSQL(
    uast.WithDialect(uast.DialectMariaDB)
)
defer builder.Close()
mariadbQuery, mariadbArgs, _ := builder.Build(stmt)
builder.SetDialect(uast.DialectMsSQL)
mssqlQuery, mssqlArgs, _ := builder.Build(stmt)
builder.SetDialect(uast.DialectMySQL)
mysqlQuery, mysqlArgs, _ := builder.Build(stmt)
builder.SetDialect(uast.DialectPostgreSQL)
postgresqlQuery, postgresqlArgs, _ := builder.Build(stmt)
builder.SetDialect(uast.DialectSQLite)
sqliteQuery, sqliteArgs, _ := builder.Build(stmt)

```

Output MariaDB:

```

SELECT `u`.`string` FROM `users` AS `u` WHERE `u`.`id` = ?

```

Output MsSQL:

```

SELECT [u].[string] FROM [users] AS [u] WHERE [u].[id] = @p1

```

Output MySQL:

```

SELECT `u`.`string` FROM `users` AS `u` WHERE `u`.`id` = ?

```

Output PostgreSQL:

```

SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = $1

```

Output SQLite:

```
SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = ?
```

WithMutate/SetMutate

WithMutate

Build()

Build()

SetMutate

SetDialect

```

stmt1 := uast.NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "id"), uast.Value(1)),
    )
immutableBuilder := uast.NewSQL(
    uast.WithDialect(uast.DialectPostgreSQL),
)
defer immutableBuilder.Close()
query1, _, _ := immutableBuilder.Build(stmt1)
query2, _, _ := immutableBuilder.Build(stmt1)
immutableBuilder.SetMutate(true)
query3, _, _ := immutableBuilder.Build(stmt1)
query4, _, _ := immutableBuilder.Build(stmt1)
stmt2 := uast.NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "id"), uast.Value(1)),
    )
stmt3 := uast.NewSelect(uast.NewTable("users").As("u")).
    Field(
        uast.Field[string]("u", "string"),
    ).
    Where(
        uast.Equal(uast.Field[int]("u", "id"), uast.Value(1)),
    )
mutableBuilder := uast.NewSQL(
    uast.WithDialect(uast.DialectPostgreSQL),
    uast.WithMutate(true),
)
defer mutableBuilder.Close()
query5, _, _ := mutableBuilder.Build(stmt2)
query6, _, _ := mutableBuilder.Build(stmt2)
mutableBuilder.SetMutate(false)
query7, _, _ := mutableBuilder.Build(stmt2)
query8, _, _ := mutableBuilder.Build(stmt3)

```

Output Query1:

```

SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = $1

```

Output Query2:

```
SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = $1
```

Output Query3:

```
SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = $1
```

Output Query4:

```
// Undefined result – stmt was mutated
```

Output Query5:

```
SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = $1
```

Output Query6:

```
// Undefined result – stmt was mutated
```

Output Query7:

```
// Undefined result – stmt was mutated
```

Output Query8:

```
SELECT "u"."string" FROM "users" AS "u" WHERE "u"."id" = $1
```
